UNIVERZITA KOMENSKÉHO V BRATISLAVE

FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY

PRIESKUMNÍK GRAFOVÝCH ŠTRUKTÚR PRE LOGIKU PRVÉHO RÁDU

BAKALÁRSKA PRÁCA

2020 Miroslav Baluch

UNIVERZITA KOMENSKÉHO V BRATISLAVE

FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY

PRIESKUMNÍK GRAFOVÝCH ŠTRUKTÚR PRE LOGIKU PRVÉHO RÁDU

BAKALÁRSKA PRÁCA

Študijný program:	Aplikovaná informatika
Študijný odbor:	Aplikovaná informatika
Školiace pracovisko:	Katedra aplikovanej informatiky
Školiteľ:	Mgr. Ján Kľuka, PhD.

Bratislava 2020

Miroslav Baluch

52087286

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Miroslav Baluch
Študijný program: aplikovaná informatika (Jednoodborové štúdium, bakalársky I. st., denná forma)
Študijný odbor: Informatika
Typ záverečnej práce: Bakalárska
Jazyk záverečnej práce: Slovenský
Sekundárny jazyk: Anglický
Názov: Prieskumník grafových štruktúr pre logiku prvého rádu
Explorer of graph structures for first-order logic

Anotácia: V roku 2017/18 v rámci bakalárskej práce Milana Cifru vznikol výučbový nástroj nazvaný Prieskumník sémantiky logiky prvého rádu. Nástroj umožňuje definovať prvorádové jazyky, štruktúry interpretujúce ich symboly a vyhodnocovať termy a formuly. Štruktúry je možné vytvárať definovaním v množinovej notácii a pomocou matíc. Pre lepšie pochopenie štruktúr a ich súvislosti s praktickými problémami je však vhodné prieskumník ďalej rozšíriť o grafový a prípadne aj databázový pohľad na štruktúry.

Cieľ: Rozšíriť existujúci prieskumník sémantiky logiky prvého rádu o grafovú vizualizáciu a editovanie štruktúr a prípadne aj zobrazenie a vizualizáciu v štýle databázových tabuliek.

Literatúra: Barker-Plummer, D., Barwise, J., Etchemendy, J.: Language, Proof and Logic. Second Edition. CSLI Publications, 2011.
Cifra, M.: Prieskumník sémantiky logiky prvého rádu. Bakalárska práca – Univerzita Komenského, Bratislava, 2018.

Kľúčové slová: logika prvého rádu, splnenie v konečnej štruktúre, webová aplikácia na strane klienta

Vedúci: Mgr. Ján Kľuka, PhD.
Konzultant: RNDr. Jozef Šiška, PhD.
Katedra: FMFI.KAI - Katedra aplikovanej informatiky
Vedúci katedry: prof. Ing. Igor Farkaš, Dr.

Dátum zadania: 04.10.2019

Dátum schválenia: 07.10.2019

doc. RNDr. Damas Gruska, PhD.
garant študijného programu

....

student vedúci práce

Poďakovanie: Chcel by som sa touto cestou poďakovať školiteľovi mojej práce, Mgr. Jánovi Kľukovi, PhD. Ďakujem mu za čas, ktorý mi venoval pri písaní tejto práce. Veľmi si cením jeho rady, ochotu a trpezlivosť. Zároveň by som sa chcel poďakovať študentom, ktorí mi pomohli s testovaním mojej aplikácie.

Abstrakt

Práca sa zaoberá rozšírením aplikácie Prieskumník sémantiky logiky prvého rádu o nové súčasti, ktoré predstavujú úplne iný pohľad na precvičovanie štruktúr logiky prvého rádu ako doteraz ponúkala pôvodná aplikácia. Aplikácia je rozšírená hlavne o grafový pohľad, v ktorom sú prvky domény, symboly konštánt, ternárne a kvaternárne vzťahy vyjadrené pomocou vrcholov grafu a binárne vzťahy sú vyjadrené pomocou hrán. Následne je možné si definovať výrazy (formuly a termy), ktoré aplikácia vyhodnocuje podľa vytvorenej štruktúry. Pôvodný pohľad na štruktúry je v aplikácii zachovaný a je možné ľahko prepínať medzi starým a novým pohľadom. Na to bolo nutné zabezpečiť konzistenciu údajov medzi pohľadmi. Aplikácia je primárne naprogramovaná pomocou knižníc React, Redux a React Diagrams. Bola testovaná študentami predmetu Matematika 4 – Logika pre informatikov.

Kľúčové slová: logika prvého rádu, splnenie v konečnej štruktúre, webová aplikácia na strane klienta

Abstract

The work deals with the extension of the Explorer of Semantics of First-Order Logic with new components, which represent a completely different view of practicing the structures of first-order logic than the original application offered so far. The application is extended mainly by a graph view, in which the elements of the domain, symbols of constants, ternary and quaternary relations are expressed by means of graph vertices and binary relations are expressed by means of edges. Subsequently, it is possible to define expressions (formulas and terms), which the application evaluates according to the created structure. The original view of the structures is preserved in the application and it is possible to easily switch between the old and the new view. To do this, it was necessary to ensure consistency of data between views. The application is primarily programmed using the React, Redux and React Diagrams libraries. It was tested by students of the subject Mathematics (4) - Logic for computer science.

Keywords: first-order logic, satisfaction in finite structure, client-side web application

Obsah

Úvod 1
1 Logika prvého rádu 3
1.1 Syntax 3
1.2 Sémantika 4
2 Použité technológie 6
2.1 Základné webové technológie 6
2.2 React 6
2.3 Redux 9
2.4 TypeScript 11
2.5 Bootstrap a Font Awesome 12
2.6 Grafový editor 12
2.6.1 React Diagrams 12
2.6.2 React Digraph 15
3 Podobné práce 17
3.1 Prieskumník sémantiky logiky prvého rádu 17
3.2 Kolaboratívny grafický editor pre MediaWiki 18
4 Prieskumník sémantiky logiky prvého rádu 19
4.1 Organizácia kódu 19
4.2 Štruktúra stavu 19
4.2.1 Redux 21
5 Analýza problému 22
5.1 Zámer aplikácie 22
5.2 Funkčné požiadavky na aplikáciu 22
5.3 Ostatné požiadavky na aplikáciu 25
6 Návrh 26
6.1 Výber grafového editora 26
6.2 Vývoj aplikácie 26
6.2.1 Prvý návrh 26
6.2.2 Druhý návrh 27
6.2.3 Tretí, finálny návrh 27
6.3 Návrh vzhľadu vrcholov 28
6.3.1 Doménový vrchol 28
6.3.2 Vrchol konštanty 29
6.3.3 Binárny vzťah 30
6.3.4 Ternárny a kvaternárny vrchol 30

6.4 Dátový model 31
6.5 Používateľské rozhranie 31
6.5.1 Návrh pridávania vrcholov do grafu 34
6.6 Import/Export 34
7 Implementácia 35
7.1 Použitý jazyk 35
7.2 Refaktorizácia existujúcej aplikácie 35
7.3 Migrácia na novší Bootstrap 35
7.4 Organizácia kódu do balíkov 36
7.5 Organizácia komponentov a kontajnerov 36
7.6 Organizácia stavu aplikácie 38
7.6.1 Štruktúra stavu aplikácie 39
7.6.2 Aktualizovanie objektov grafu pri zmene ich stavu 39
7.6.3 Akcie a reducery 40
8 Testovanie so študentami 42
8.1 Zámer testovania 42
8.2 Priebeh testovania 42
8.3 Výsledky testovania – chyby 42
8.4 Výsledky testovania – spätná odozva 42
8.5 Záver testovania 43
Záver 44
Literatúra 45
Príloha A - Dotazník 46
Príloha B – Úloha z logiky 47
Príloha C – Elektronická príloha 48

Úvod

Vzdelávanie je veľmi dôležitým aspektom dnešnej doby. Tá sa vyznačuje rýchlym vývojom a častými zmenami. Na zachytenie tohto trendu vzniká množstvo webových nástrojov a služieb, ktoré sa snažia uľahčiť, resp. zjednodušiť proces výučby. Aplikácie tvorené pomocou webových technológií sú ľahko prístupné, takmer vždy dostupné a väčšinou nevyžadujú od používateľa, aby vlastnil najvýkonnejší hardware. Aj kvôli tomu je mojou snahou vytvoriť webovú aplikáciu, ktorá má uľahčiť učenie a pochopenie logiky prvého rádu na škole.

Moja práca bude vychádzať z existujúcej bakalárskej práce Milana Cifru, *Prieskumník štruktúr* [1]. Budem pracovať s kódom tejto existujúcej práce a využijem ho na rozšírenie a vytvorenie komplexnej aplikácie. Momentálna aplikácia predstavuje tzv. matematický (množinový) pohľad na logiku. Používateľ si v nej zadefinuje konečnú štruktúru pre definovaný jazyk. Štruktúra sa definuje vymenovaním prvkov množín, ktoré ju tvoria – domény a interpretácií predikátových a funkčných symbolov. Zároveň používateľ v tomto jazyku vytvára výrazy, ktoré predstavujú formuly a termy. Tieto výrazy systém vyhodnocuje, či sú formuly splnené, resp. akú hodnotu nadobúda term. Nakoľko je používateľ okamžite informovaný o stave vyhodnotenia, tak si vie ľahko a rýchlo opraviť chybu. Zároveň bolo cieľom študenta Milana Cifru urobiť taký systém, ktorý nerobí všetko za používateľa automaticky, ale program musí najprv dostať vstup od používateľa, ktorý následne vyhodnotí.

Matematický pohľad na štruktúry je presný, ale dostatočne neuľahčuje vnímanie a pochopenie štruktúr. Preto je cieľom mojej bakalárskej práce poskytnúť interaktívny grafový a databázový pohľad na logické štruktúry. O tieto možnosti rozšírim existujúci webový výučbový nastroj. Viaceré pohľady majú pomôcť študentom vybrať si ideálny spôsob učenia.

Grafový pohľad predstavuje úplne iný pohľad na logiku ako matematický a databázový. Ide o grafické zobrazenie, v ktorom je možné pomocou objektov grafu vyjadriť to, čo v matematickom pohľade. Základnou myšlienkou je, že prvky domény predstavujú vrcholy grafu a dvojice z interpretácií binárnych predikátov sú hrany. Konkrétne zobrazenie jednotlivých častí logiky predstavuje problém pri vývoji mojej aplikácie, ktorý budem v práci riešiť. Snahou bude vytvoriť aplikáciu, ktorá bude pre používateľov intuitívna a bude sa ľahko používať.

Databázový pohľad predstavuje zobrazenie predikátových a funkčných symbolov logiky prvého rádu v štýle relačných databáz. Keďže interpretácie predikátových a funkčných symbolov sú taktiež relácie, tak tento pohľad je veľmi vhodný na ich reprezentáciu. Okrem toho sa relačné databázy vyučujú v tomto študijnom programe.

Obsah textu je rozdelený na kapitoly. V prvej kapitole objasním základy logiky, ktorá predstavuje základ pre aplikáciu. Budem sa riadiť definíciami logiky. V druhej kapitole sa venujem technológiám, ktoré som použil na rozšírenie existujúcej webovej aplikácie. Následne opíšem grafické editory, ktoré sme skúmali za účelom nájdenia najvhodnejšieho editora, použitého v rámci mojej aplikácie. Okrem toho vysvetlím prečo sme sa rozhodli využiť práve vybratý editor. Následne sa v ďalšej kapitole pozriem na už podobné, existujúce práce. V tejto kapitole spomeniem aj prácu, na ktorú nadväzuje moja práca. V kapitole 4 presnejšie opíšem štruktúru kódu existujúcej aplikácie, aby bolo zreteľné, ktoré časti aplikácie boli už predom naprogramované. V kapitole 5 detailne vysvetlím zámer mojej aplikácie a popíšem funkčné požiadavky samotnej aplikácie. Následne sa budem venovať podrobne implementácii, ako je

kód mojej aplikácie štruktúrovaný a spomeniem aj neštandardné veci, ktoré predstavovali výzvu pri vývoji aplikácie. Posledná kapitola je venovaná súhrnu testovania aplikácie, ktoré bolo vykonané so študentami predmetu *Matematika 4 – Logika pre informatikov*.

1 Logika prvého rádu

V prvej kapitole vysvetlím základné pojmy logiky prvého rádu. Spojím dokopy formálny a neformálny štýl, aby boli vysvetlené pojmy zrozumiteľné aj pre ľudí, ktorí sa s logikou doteraz nestretli. Definície vychádzajú z prednášok predmetu Matematika (4) – Logika pre informatikov [2] a Švejdarovej učebnice logiky [3].

Logika prvého rádu tvorí rodinu formálnych jazykov a predstavuje zjednodušenie prirodzeného jazyka. Tieto jazyky slúžia na zápis tvrdení, ktoré hovoria o vlastnostiach a vzťahoch objektov. Jazyk logiky prvého rádu je tvorený pevne zvolenou množinou symbolov, ktoré sa používajú na vytváranie formúl a termov. Delíme ho na syntax a sémantiku.

1.1 Syntax

Syntax predstavuje pravidlá, ktorými určujeme, ktoré postupnosti symbolov sú správne vytvorené. Zároveň určuje množiny symbolov, ktoré sa môžu vyskytovať v týchto postupnostiach. Symboly jazyka sa delia na (individuové) premenné, mimologické, logické a pomocné symboly.

Symboly (indivíduových) premenných sú symboly z nejakej nekonečnej spočítateľnej množiny, definíciou označenej ako $V_{\mathcal{L}}$. Tieto symboly sú označené malými písmenami z konca abecedy (*x*, *y*, ...). Používame ich na vyjadrenie niekoho, niečoho, teda na označovanie objektov. Ich presný význam závisí od kvantifikátorov, ktoré ich viažu. Množina premenných môže vyzerať nasledovne: $V_{\mathcal{L}} = \{\mathrm{x}_0, \mathrm{x}_1, \mathrm{x}_2, \dots\}$.

Ďalšou skupinou sú *mimologické symboly*, medzi ktoré patria *symboly konštánt,* ktoré sú z nejakej spočítateľnej množiny $C_{\mathcal{L}}$, *funkčné symboly* $F_{\mathcal{L}}$ a *predikátové symboly* $P_{\mathcal{L}}$.

Symboly konštánt sú symboly z nejakej spočítateľnej množiny $C_{\mathcal{L}}$ pre jazyk $\mathcal{L}$. Konštanta označuje jeden konkrétny objekt, priradený z domény cez interpretačnú funkciu. Množina konštánt môže vyzerať napríklad takto: $C_{\mathcal{L}} = \{\mathrm{a}, \mathrm{b}\}$.

Predikátové symboly z množiny $P_{\mathcal{L}}$ pre jazyk $\mathcal{L}$ väčšinou vyjadrujú nejakú (unárnu) vlastnosť alebo vzťah dvoch alebo viacerých objektov. Vlastnosť môže byť vysoký(Ján), ktorá vyjadruje, že Ján je vysoký, avšak bližšie nešpecifikuje jeho výšku. V zložitejších prípadoch, ak by sme chceli napríklad vyjadriť vzťah objektu a výšky, by to vyzeralo nasledovne: vysoký(Ján, 185), ktorý vyjadruje, že Ján (objekt) je vysoký 185 (výška). Množina predikátových symbolov môže vyzerať nasledovne: $P_{\mathcal{L}} = \{\mathrm{vysoký}^1, \mathrm{vyhral}^2\}$.

Funkčné symboly z množiny $F_{\mathcal{L}}$ pre jazyk $\mathcal{L}$ označujú funkcie, v ktorých každému prvku, ktorý predstavuje n-ticu priraďujeme práve jednu hodnotu. Zapisujeme ich v tvare symbol(n-tica). Okrem toho musí pre každú funkciu platiť, že je všade definovaná. Teda pre každú funkciu a každú n-ticu existuje nejaká, práve jedna hodnota.

Majme funkčný symbol hodnotený, ktorý vyjadruje, že žiak (prvý parameter n-tice) je hodnotený z predmetu (druhý parameter n-tice) známkou (hodnota n-tice). Ako príklad uvediem: hodnotený(Ján, telesná výchova, A), ktorý vyjadruje, že Ján (žiak) je hodnotený z predmetu telesná výchova (predmet) známkou A (známka). Horný index pri funkčnom symbole predstavuje aritu funkčného symbolu. Množina funkčných symbolov môže vyzerať nasledovne: $F_{\mathcal{L}} = \{\mathrm{značka}^1, \mathrm{hodnotený}^2\}$.

Následne z premenných, symbolov konštánt a funkčných symbolov vytvárame postupnosti symbolov, ktoré sa nazývajú termy. Každý symbol premennej x z $V_{\mathcal{L}}$ je term, každý symbol konštanty c z $C_{\mathcal{L}}$ je term a ak f je funkčný symbol s aritou n a $t_1, \dots, t_n$ sú termy, tak aj $f(t_1, \dots, t_n)$ je termom. Nič iné medzi termy nepatrí.

Rovnostný atóm jazyka $\mathcal{L}$ je každá postupnosť symbolov $t_1 = t_2$, kde t_1 a t_2 sú termy. Predikátový atóm jazyka $\mathcal{L}$ je každá postupnosť symbolov $P(t_1, \dots, t_n)$, kde P je predikátový symbol s aritou n a $t_1, \dots, t_n$ sú termy. Atomickými formulami jazyka $\mathcal{L}$ nazývame súhrne všetky rovnostné a predikátové atómy.

Všetky atomické formuly sú formulami. Ak A je formula tak aj $\neg A$ je formula. Ak A a B sú formuly, tak aj $(A \wedge B)$, $(A \vee B)$, $(A \rightarrow B)$ sú formuly. Ak x je indivíduová premenná a A je formula, tak aj $\exists x A$ a $\forall x A$ sú formuly.

Ďalej nasledujú *logické symboly*, ktoré používame v kombinácií s ostatnými druhmi symbolov na vytvorenie komplexnejších formúl. Medzi *logické symboly* patria *logické spojky, symbol rovnosti* a *kvantifikátory. Logické spojky* sú *unárne a binárne.*

Funkčné a predikátové symboly majú zároveň tzv. aritu, ktorá predstavuje počet argumentov (symbolov), ktoré dosádzame do funkčného alebo predikátového symbolu.

1.2 Sémantika

Následne sa bližšie pozrieme k sémantike logiky.$\mathcal{L}$ predstavuje jazyk logiky prvého rádu. Štruktúrou pre jazyk $\mathcal{L}$ je dvojica $\mathcal{M} = (M, i)$, kde M je doména štruktúry a i je interpretačná funkcia. Doména štruktúry je ľubovoľná neprázdna množina a interpretačná funkcia je zobrazenie. Interpretačná funkcia priraďuje symbolu konštanty prvok z domény M. Konštante smieme priradiť maximálne jeden prvok. Každému funkčnému symbolu f s aritou n priraďuje funkciu $i(f): M^n \rightarrow M$. Každému predikátovému symbol P s aritou n priraďuje množinu $i(P) \subseteq M^n$.

Ohodnotenie premenných e každej indivíduovej premennej priraďuje prvok z domény. *Hodnota termu t* v štruktúre $\mathcal{M}$ pri ohodnotení premenných e je prvok z M označovaný $t^{\mathcal{M}}[e]$ a definovaný nasledovne:

$x^{\mathcal{M}}[e] = e(x)$, keď x je premenná;

$a^{\mathcal{M}}[e] = i(a)$, keď a je konštanta;

$\left(f(t_1, \dots, t_n)\right)^{\mathcal{M}}[e] = i(f)(t_1^{\mathcal{M}}[e], \dots, t_n^{\mathcal{M}}[e])$, kde $t_1, \dots, t_n$ sú termy.

Vzťah štruktúra $\mathcal{M}$ *spĺňa* formulu A pri ohodnotení e (skrátene $\mathcal{M} \vDash A\,[e]$) má nasledovnú induktívnu definíciu:

$\mathcal{M} \vDash t_1 = t_2[e]$ vtt $t_1^{\mathcal{M}}[e] = t_2^{\mathcal{M}}[e]$

$\mathcal{M} \vDash P(t_1, \dots, t_n)[e]$ vtt $\left(t_1^{\mathcal{M}}[e], \dots, t_n^{\mathcal{M}}[e]\right) \in i(P)$

$\mathcal{M} \vDash \neg A[e]$ vtt $\mathcal{M} \nvDash A[e]$

$\mathcal{M} \vDash (A \wedge B)[e]$ vtt $\mathcal{M} \vDash A[e]$ a zároveň $\mathcal{M} \vDash B[e]$

$\mathcal{M} \vDash (A \vee B)[e]$ vtt $\mathcal{M} \vDash A[e]$ alebo $\mathcal{M} \vDash B[e]$

$\mathcal{M} \vDash (A \rightarrow B)[e]$ vtt $\mathcal{M} \nvDash A[e]$ alebo $\mathcal{M} \vDash B[e]$

$\mathcal{M} \vDash \exists x A[e]$ vtt pre nejaký prvok $m \in M$ máme $\mathcal{M} \vDash A[e(x/m)]$

$\mathcal{M} \vDash \forall x A[e]$ vtt pre každý prvok $m \in M$ máme $\mathcal{M} \vDash A[e(x/m)]$

2 Použité technológie

V tejto kapitole stručne vysvetlím (hlavné) použité technológie na vytvorenie aplikácie a pridám k ním aj krátke ukážky.

2.1 Základné webové technológie

V mojej bakalárskej práci budú použité základné webové technológie, ktoré slúžia na tvorbu webových stránok. Patrí sem JavaScript, NPM, HTML a CSS. JavaScript [4] je skriptovací interpretovaný programovací jazyk. Používa sa primárne pri tvorbe interaktívnych webových stránok. V tomto projekte nebudeme používať základný (natívny) JavaScript, ale JavaScriptovú knižnicu nazývanú React.

Na inštaláciu knižníc (balíkov) budem používať nástroj Node package manager (v skratke npm). Pomocou tohto softwaru je možné nainštalovať všetky potrebné balíky z jedného miesta, teda nie je nutné pristupovať ku každému balíku samostatne.

Základná štruktúra stránky bude vytvorená prostredníctvom jazyka HTML. Ide o [5] značkovací jazyk pozostávajúci z textu a značiek (tagov), ktorými určujeme význam a štruktúru webových stránok. Značky vyjadrujú vlastnosti textu a sú zvyčajne párové. Časť textu medzi otváracou a zatváracou značkou sa nazýva element. Elementy je možné do seba vnárať, čím vytvárajú stromovú štruktúru. Tieto značky slúžia k rozdeleniu textu na časti - bloky (ako sú odsek, nadpis, hlavná časť a pod.). Ďalšími značkami je možné tieto časti textu samostatne formátovať. Veľkou výhodou HTML jazyka je čitateľnosť pre ľudí aj stroje.

Na menenie vizuálnej stránky HTML dokumentov, určovanie polohy elementov na stránke a pre jednoduché akcie vykonané pri interakcií s používateľom (ako je zvýraznenie textu po nadídení naň a pod.) budem používať kaskádové štýly [6] (v skratke nazývané CSS).

2.2 React

React [7] predstavuje knižnicu pre JavaScript, zameranú na ľahšie vytváranie používateľských rozhraní. Momentálne je spravovaný spoločnosťou Facebook. Výhodou Reactu oproti iným nástrojom je možnosť rozdeliť používateľské rozhranie aplikácie na komponenty. Komponent predstavuje základný prvok knižnice. Typovo predstavuje triedu alebo funkciu. Zároveň je možné z menších komponentov vytvárať väčšie a každý z komponentov sa dá využiť ako samostatná, znovu použiteľná časť.

React komponent funguje ako virtuálny element. Zoskupuje viacero elementov a HTML textových uzlov (značiek), poprípade iných komponentov, ktoré dokopy tvoria logický celok. Nasledujúcou cestou je možné vytvoriť React komponent:

```
const component = React.createElement('div',{},'Bakalárska Práca');
```

Kód 2.1: Vytvorenie React komponentu štandardnou cestou

Avšak veľkou výhodou Reactu je vytváranie HTML kódu pomocou rozšírenia JSX. JSX predstavuje syntaktické rozšírenie samotného JavaScriptu. Dovoľuje písať HTML elementy v rámci JavaScriptu, čím je kód pre programátora čitateľnejší a zároveň samotné vytváranie aplikácie je jednoduchšie. Uľahčuje kódovanie tým, že metódy ako `createElement` alebo `appendChild` nie je vôbec potrebné použiť. Kód v JSX sa následne konvertuje do obyčajného JavaScriptu cez prekladač. Ten istý komponent, ktorý bol vytvorený cez metódu `createElement` je možné vytvoriť nasledovne:

```
const component = () => (
    <div>Bakalárska práca</div>
);
```

Kód 2.2: Vytvorenie React komponentu pomocou rozšírenia JSX

V príklade kódu 3.2 išlo typovo a funkciu. Vytvorenie React triedy je o niečo zložitejšie, avšak dovoľuje aby mal komponent svoj vlastný stav, ktorý je prístupný cez atribút `state`. Hodnotou tohto atribútu je objekt, ktorý má ľubovoľné, programátorom definované atribúty. Tento objekt si triedny komponent riadi sám. Pri zmene atribútu `state` nastane prekreslenie (prerenderovanie) komponentu. Aby bol dosiahnutý čo najlepší výkon, tak React neprekresľuje komponent pri úplne každej zmene, ale nazhromaždí viacero zmien a potom následne komponent raz prekreslí. Teda aktualizovanie React komponentu je asynchrónne, čo znamená, že aktualizovanie sa nevykonáva okamžite, a teda aj pri zmene hodnoty premennej môže aj tak nastať prípad, že stále pracujeme so starou hodnotou premennej. Atribút `state` sa inicializuje v konštruktore triedneho komponentu:

```
this.state = {
      modalShow: false,
      diagramToggled:true,
      exerciseName:''
    };
```

Kód 2.3: Inicializovanie React atribútu `state`

Následne je možné v kóde meniť stav premenných state pomocou funkcie `setState`:

```
this.setState({diagramToggled:bool})
```

Kód 2.4: Zmenenie stavu komponentu pre premennú `diagramToggled`

State nie je v Reacte jediným atribútom triedneho komponentu. Okrem neho ešte existuje atribút `props`. Obidva tieto atribúty sú zodpovedné za zmenu stavu komponentu, ktorá následne spôsobí renderovanie samotného komponentu. Každý z týchto atribútov predstavuje obyčajný JavaScriptový objekt, ale nesúci v sebe jeden základný rozdiel. Atribút `props` sa dostane do komponentu z rodičovského komponentu a sú v rámci dieťaťa nemenné (sú `immutable`). Konštruktor dieťaťa berie ako parameter `props`. Následne je ešte nutné v rámci konštruktora zavolať rodičovskú triedu s parametrom `props` cez `super(props)`. Komponent, ktorý pracuje s `props` ich iba spracuje, ale nemení (môže ich posunúť do ďalšieho komponentu). `Props` môže reálne meniť iba rodič komponentu (teda komponent, ktorý je najvyššie v hierarchii, cez ktorého boli `props` vytvorené). Ako už bolo vyššie spomenuté, `state` sa ovláda v rámci komponentu, kde je vytvorený. Ak chceme meniť stav komponentu v jeho podkomponente, tak je nutné definovať samostatnú funkciu, ktorú je nutné naviazať (`bind`) na aktuálny komponent. Následne je už možné poslať referenciu na funkciu cez `props` a meniť ňou stav rodičovského komponentu.

```
constructor(props){
    super(props);
    this.state = {
        modalShow: false,
        diagramToggled:true,
        exerciseName:''
    };
    this.setModelShowState = this.setModelShowState.bind(this);
}
setModelShowState(bool:boolean){
    this.setState({modalShow: bool});
}
```

Kód 2.5: Ukážka konštruktora s parametrom `props` a naviazanie funkcie v komponente

Po naviazaní je možné funkciu volať aj z iných komponentov. Okrem toho, ak je typ komponentu trieda, tak takýto komponent musí byť podtriedou (`extends`) triedy `React.Component` a musí v sebe obsahovať metódu `render`, ktorá slúži na zobrazenie (renderovanie) samotného komponentu.

V prípade komponentu implementovaného funkciou stačí vrátiť JSX kód cez príkaz `return`. JSX kód musí byť obalený, čo znamená, že na najvyššej úrovni môže byť maximálne jeden HTML element. To znamená, že nemožno vrátiť:

```
return (
    <div>Bakalárska práca</div>
    <div></div>
)
```

Kód 2.6: Nesprávny `return`

A zároveň taktiež nie je možné vrátiť prázdny komponent. Poslednou základnou vecou, ktorú chcem spomenúť v rámci Reactu je vytváranie a používanie udalostí. Udalosť predstavuje interakciu používateľa s používateľským rozhraním, pričom sa niečo vykoná. Jednou z udalostí je napríklad `onClick`:

```
<button title={"Pridaj"} onClick={() => onClickAdd(addType)}>Pridaj</button>
```

Kód 2.7: Tlačidlo s `onClick` udalosťou

Po kliknutí na tlačidlo sa vykoná akcia `onClickAdd`.

Na koniec tejto sekcie uvediem dva komplexnejšie príklady React komponentov, ktoré zobrazujú tlačidlo s atribútmi variant a title a nápisom `"+ Pridaj"`, ktoré reaguje na kliknutie vyvolaním funkcie `addFunction`. Vytvorený element (v tomto prípade tlačidlo) je v obidvoch príkladoch rovnaký, líši sa iba v type komponentu, ktorým bol vytvorený. Prvý príklad je vytvorený pomocou funkčného komponentu, zatiaľ čo v druhom príklade je tlačidlo vytvorené pomocou triedneho komponentu.

```
const AddButton = ({addFunction,addType}) => (
    <Button variant={"success"} title={"Pridaj"}
            onClick={() => addFunction(addType)}>
        <FontAwesome name='plus'/>
         Pridaj
    </Button>
);
```

Kód 2.8: Tlačidlo vytvorené cez komponent typu funkcia

```
class AddButton extends React.Component {
    constructor(props) {
        super(props);
    }
    render() {
        return(
            <Button variant={"success"} title={"Pridaj"}
                    onClick={() => this.props.addFunction(this.props.addType)}>
                <FontAwesome name='plus'/> Pridaj
            </Button>
        )
    }
}
```

Kód 2.9: Tlačidlo vytvorené cez triedny komponent

2.3 Redux

Redux [8] je JavaScriptová knižnica používaná na riadenie stavu premenných, teda aké hodnoty premenné nadobúdajú. Využíva sa napríklad v kombinácií s Reactom. Všetky dáta (resp. stav aplikácie), ktoré aplikácia spracováva sa zhlukujú pod jeden JavaScriptový objekt, nazývaný aj store – úložisko. Tým pádom je možné pristupovať ku všetkým premenným z jedného miesta, čo je oveľa prehľadnejšie a práca s dátami je teda jednoduchšia.

Redux teda predstavuje model, štruktúru, ktorou riadime prístup, menenie, pridávanie a mazanie dát. V rámci Reduxu nie je možné priamo meniť stav premenných, aby sa predišlo chybám. Na menenie hodnôt slúžia akcie. *Akciu* predstavuje obyčajný JavaScriptový objekt, ktorý popisuje, čo sa udialo. Akcia musí obsahovať atribút `type`. Tento atribút indikuje typ akcie, ktorá bola vykonaná. Vďaka tomu presne vieme čo sa v aplikácii udialo a čím bola hodnota zmenená. Okrem toho môžeme do akcie zadefinovať dodatočné parametre (ak ich potrebujeme).

```
export const action1 = (count) =>({
  type: 'ACTION1',
  count
});
```

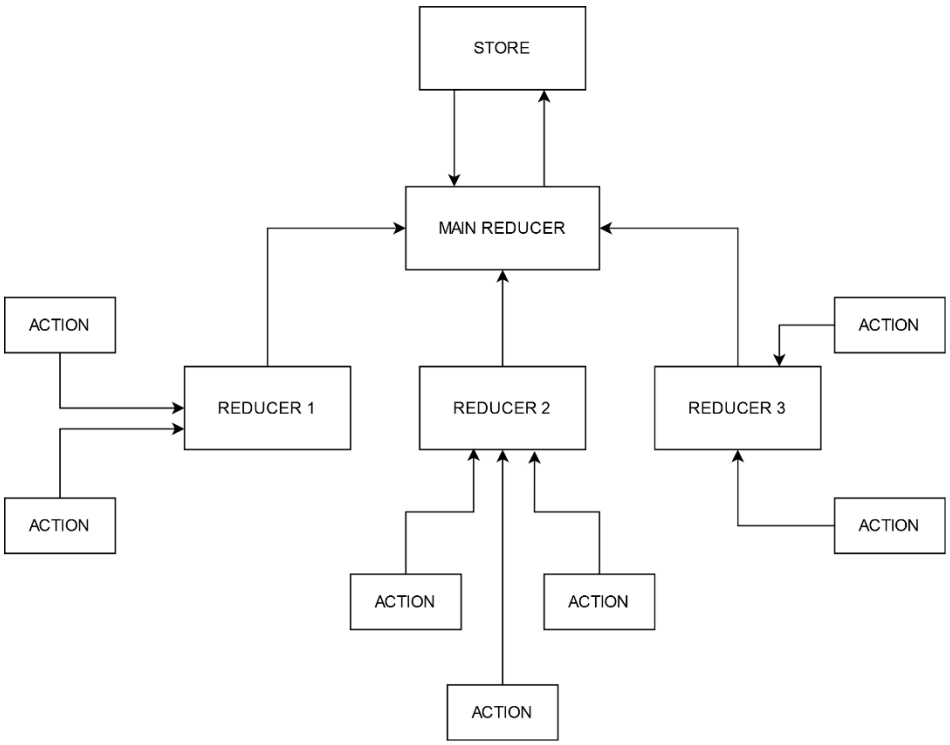
Kód 2.10: Jednoduchá akcia

Avšak samotná akcia nemení stav. Na to slúži funkcia nazývaná *reducer*. Prvý parameter tejto funkcie je stav úložiska. Ide buď o základný stav (to je stav po vytvorení úložiska) alebo predchádzajúci stav (pretože momentálny stav bude ten, ktorý dostaneme po zavolaní reducer funkcie). Druhý parameter je samotná akcia, ktorá sa má vykonať. Samotný reducer vykonáva akciu tým, že mení stav. Pre správne aktualizovanie Redux stavu je avšak nutné, aby výsledkom vykonanej akcie bol nový stav, nie modifikácia pôvodného stavu. Má ísť o kópiu pôvodného stavu. Obzvlášť je to dôležité v kombinácií s Reactom pri tých komponentoch, ktoré berú údaje z Redux stavu, pretože to ovplyvňuje renderovanie takýchto komponentov (pri zmene props sa komponent prerenderuje). Redux je naprogramovaný tak, že predošlý a nový stav porovnáva iba povrchne a nie do hĺbky. Dôvodom je zachovanie výkonu, hlavne v prípade, keď je stav príliš veľký. Jeden reducer môže obsluhovať viacero akcií. Aby sa predišlo vedľajším efektom (napríklad pri sieťovej komunikácii) a cyklickému chovaniu (neustále volanie akcií), tak počas trvania jednej akcie nie je možné volať inú akciu. Na nasledujúcich riadkoch je kód na vytvorenie jednoduchého reducera:

```
function reducerExample(state = initialState, action) {
    switch (action.type) {
        case 'ACTION1':
         return {
            ...state,
            count: action.count+1
         };
        case 'ACTION2':
            let count = action.count;
         return {
            ...state,
            count: action.count-1
         };
        default:
            return state;
    }
}
```

Kód 2.11: Vytvorenie reducera s tromi akciami

Väčšinou sa stav skladá z viacerých menších častí, pre ktoré môžeme definovať viacero reducerov, pričom každý z nich spravuje určitú časť stavu. Nakoniec spojíme všetky reducery dokopy a tým vytvoríme jeden celistvý. Na vytvorenie štruktúry, ktorá sleduje stav premenných voláme metódu `createStore`, ktorá berie ako parameter reducer. Tomuto parametru väčšinou dáme hlavný, spájajúci reducer. Nasledovný obrázok približuje ako jednotlivé časti Reduxu medzi sebou komunikujú.



Obr. 2.1: Opis práce Reduxu

2.4 TypeScript

TypeScript [9] predstavuje nadstavbu pre JavaScript, pridávajúcu typový systém. Týmto je ochrana pred zlým vstupom alebo problémom oveľa kontrolovateľnejšia, keďže nie je možné v kóde priradiť premennej iný typ ako môže nadobúdať. Ak teda máme napríklad premennú typu integer, tak táto premenná môže nadobudnúť iba hodnotu typu integer, nie iný typ.

V natívnom JavaScripte programátor pri deklarácií premenných neurčuje typ. Avšak jadro JavaScriptu pracuje s typmi, aby vedelo rozlíšiť, o aký objekt ide a prípadne akú funkciu má nad ním zavolať. V klasickom JavaScripte je teda možné nasledovne zadeklarovať premennú:

```
let hodnota;
hodnota = 10;
```

Kód 2.12: Vytvorenie číselnej hodnoty cez JavaScript

I keď premenná `hodnota` teraz predstavuje číslo, môžeme do nej vložiť aj iný typ:

```
let hodnota = 10;
hodnota = "Ahoj";
hodnota = { pozdrav: "Ahoj" };
```

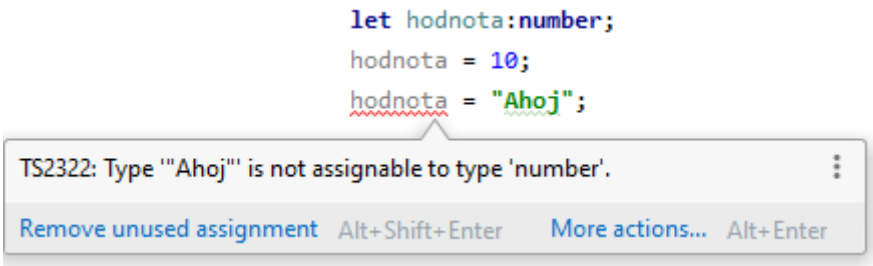
Kód 2.13: Vytvorenie hodnoty a následne pretypovanie premennej

Premennej `hodnota` sme najprv priradili číslo. Potom sme znovu tej istej premennej priradili reťazec, a nakoniec ešte objekt. JavaScript sám určil o aký typ premennej ide a pretypoval ho. Avšak v prípade použitia knižnice TypeScript toto nie je možné. Zadefinujme si teda tú istú premennú, ale s použitím TypeScriptu:

```
let hodnota:number;
hodnota = 10;
```

Kód 2.14: Vytvorenie číselnej hodnoty s použitím TypeScriptu

Premennej `hodnota` sme určili typ `number`, čo predstavuje číslo. Teda do tejto premennej môžeme dosadiť iba číslo, nič iné. Ak by sme sa pokúsili priradiť iný typ tak dostaneme chybovú hlášku:



Obr. 2.2: Chyba, ktorú TypeScript oznámi po dosadení iného typu

Táto chybová hláška hovorí, že reťazec `"Ahoj"` nemožno priradiť na miesto, kde sa očakáva číslo. Deklarované typovanie je teda hlavnou funkcionalitou TypeScriptu. Kód je vďaka pridaným typom kontrolovateľnejší.

Na koniec tejto sekcie uvediem ako komplexnejší príklad funkciu s jedným parametrom. Táto funkcia dostane na vstupe číslo, vo funkcii ho spracuje a vráti výsledok. Ak by sme ako parameter dostali reťazec alebo iný nevyhovujúci typ, tak telo funkcie nevie spracovať takúto hodnotu, keďže očakáva číslo. Tým, že pridáme typovanie, tak tento jav nemôže nastať, pretože musíme dosadiť konkrétny typ. Takto sa vieme vyhnúť potencionálnym chybám. Na druhej strane to pre programátora znamená, že sa musí naučiť pracovať s novým modulom a zároveň implementovať deklarácie do kódu, čo spomaľuje vývoj.

V prípade malých projektov nie je až tak potrebné použiť TypeScript, avšak v prípade väčších projektov je to takmer nutnosťou. Čas strávený pri písaní TypeScriptu je väčšinou kratší ako neskoršie odlaďovanie chýb spôsobené zlým typom.

2.5 Bootstrap a Font Awesome

Bootstrap [10] je framework určený na vytváranie interaktívnych webových stránok, využívajúcich primárne HTML, CSS a JS. Obsahuje sadu štýlových tried pre elementy používateľského rozhrania. Týmto triedam je možné definovať podrobnejšie vlastnosti ako sú umiestenie, veľkosť, farbu ich elementov. React Bootstrap [11] je knižnica, ktorá spája React komponenty a Bootstrap.

Obr. 2.3: Základný vzhľad tlačidla bez použitia Bootstrap štýlu

Obr. 2.4: Tlačidlo s Bootstrap štýlom (vľavo bez nadídenia myši, vpravo po nadídení myšou)

V prípade knižnice React Bootstrap, každý z komponentov začína veľkým písmenom. Okrem toho má väčšina takýchto komponentov aj dodatočné atribúty, ktoré presnejšie definujú štýl daného komponentu. Vytvorenie takýchto komponentov je rovnako jednoduché ako v Reacte:

```
<Button variant={"outline-primary"}>Add</Button>
```

Kód 2.15: Vytvorenie tlačidla so štýlom s použitím React Bootstrap

Takto sme vytvorili tlačidlo, ktoré zároveň obsahuje atribút `variant`. Tento atribút slúži na definovanie štýlu tlačidla. V prípade ak by sme nevyužili túto knižnicu, ale iba samotný Bootstrap, tak vytvorenie takéhoto tlačidla by bolo realizované pomocou React atribútu `className`:

```
<button type="button" className={"btn btn-outline-primary"}>Add</button>
```

Kód 2.16: Vytvorenie tlačidla so štýlom bez použitia knižnice React Bootstrap

Okrem toho využívam knižnicu Font Awesome [12], ktorá pridáva škálovateľné vektorové ikony použiteľné pre webové stránky. V tomto prípade tiež používam knižnicu, ktorá spája v tomto prípade React komponenty a Font Awesome ikony. Takto je možné vytvárať ikony ako komponenty, pričom tieto komponenty majú špeciálny atribút `name`, vyjadrujúci typ ikony.

```
<FontAwesome name={"fas fa-arrows-alt"}/>
```

Kód 2.17: Vytvorenie Font Awesome ikony

2.6 Grafový editor

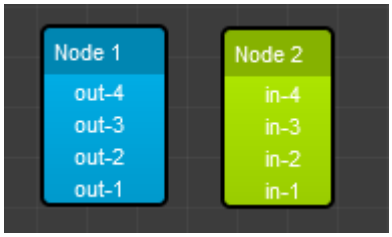
V tejto sekcii opíšem knižnice, ktoré boli kandidátom pri výbere grafového editoru pre moju aplikáciu. V sekcii 2.6.3 vysvetlím prečo som sa rozhodol práve pre daný editor spomedzi dostupných možností.

2.6.1 React Diagrams

React Diagrams [13] predstavuje modernú knižnicu, ktorá umožňuje vytvárať interaktívne diagramy. Riadi sa najnovšími normami (ES6). Objekty diagramu (časti grafu) sú tvorené

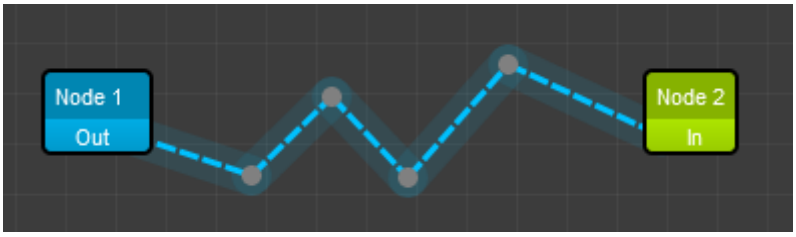
v Reacte. Zároveň využíva knižnicu TypeScript a teda chyby spojené so zlým typom sú ošetrené.

Grafy obsahujú vrcholy, hrany, porty a nadpisy. Vrcholy je možné vytvárať, mazať, klonovať a pridávať im nadpisy (v knižnici sa nadpisy nazývajú *label*). Vrchol v React Diagrams sa nazýva *node*. Vrcholu je možné definovať farbu, veľkosť, tvar a pridať im porty. *Porty* slúžia na prepojenie vrcholov a tým vznikne medzi nimi hrana. Hrana sa nazýva *link*. Samotné spojenie portov dosiahneme kliknutím alebo ťahaním myši z jedného portu do druhého. Pri týchto udalostiach (klikanie a ťahanie z portu) sa vytvorí viditeľná linka, kde jej koniec predstavuje momentálnu pozíciu myši a začiatok je port vrcholu, z ktorého sme linku ťahali. V základnej verzií grafu je možné vytvoriť hrany, ktoré nemajú koncový port, teda nikde nevedú. Hrany sú orientované, majú presne definovaný zdrojový a konečný vrchol. Medzi jednou unikátnou dvojicou vrcholov môže existovať viacero hrán (teda vytvárame duplicitné hrany). Dokonca je možné spojiť ten istý samotný vrchol. Ako som už vyššie spomenul, spájanie je realizované pomocou portov, avšak záleží o aký typ portu ide. Existujú dva typy portov, *in* a *out* port. In port je port, do ktorého smie smerovať hrana, zatiaľ čo out port, je typ portu, z ktorého je možné tvoriť hranu. Z in portu nie je možné hranu tvoriť. Teda ak chceme, aby bola práca s vrcholmi plnohodnotná, tak je potrebné, aby každý vrchol mal minimálne dva porty, pričom aspoň jeden je in a jeden je out port.



Obr. 2.5: Vzhľad vrcholov, ľavý vrchol obsahuje out porty, pravý vrchol obsahuje in porty

Nadpisy je možné vytvárať na hranách. Umiestnenie nadpisu je vždy v strede hrany. Nadpis môže obsahovať reťazec alebo emotikony. Kliknutím na hranu je zároveň možné vytvárať body (points), ktorými sa dá následne meniť tvar a dĺžka hrany. Základná časť aplikácie, kde sa umiestňujú všetky objekty grafu sa nazýva plátno (*canvas*). Plátno je interaktívne a práca nad ním je intuitívna.



Obr. 2.6: Hrana medzi dvomi vrcholmi s bodmi

Knižnica zároveň zvláda veľké množstvo objektov v grafe a výkon výrazne neklesá ani pri veľkých grafoch. Kód knižnice je veľmi dobre štruktúrovaný a modulárny. Okrem toho editor podporuje aj vytváranie vlastných typov grafových modelov (od funkcionality až po vzhľad). Vzhľad jednotlivých modelov je vytváraný cez HTML, ktoré sa dajú následne vzhľadovo upravovať pomocou kaskádových štýlov.

Momentálna verzia editora prešla výraznou zmenou oproti staršej verzii editora (kód knižnice bol prepísaný od základov) a kvôli tomu sa v nej objavujú drobné nedostatky (ako je vytváranie hrany na kliknutie spôsobí vytvorenie hrany, ktorá vedie k súradniciam (0,0)).

Okrem toho chýba detailnejšia dokumentácia. Nachádzajú sa v nej iba informácie o tom ako knižnicu nainštalovať a ako vytvoriť vlastné typy, ale je to opísane veľmi stručne. Preto je teda nutné pochopiť ako knižnica funguje analýzou kódu alebo odkazovaním sa na vytvárajúcu komunitu.

Vytvorenie grafu začína vytvorením *enginu*. Cez metódu `createEngine` vytvoríme jadro diagramu spolu s plátnom. Následne vieme nad plátnom vykonávať základné operácie ako presúvanie grafu, jednotlivých vrcholov alebo približovanie a odďaľovanie. Aby bolo možné pridať objekty grafu do plátna je nutné vytvoriť model pre graf.

```
activeModel = new DiagramModel();
```

Kód 2.18: Vytvorenie modelu

V tomto bode už môžeme vytvárať jednotlivé vrcholy. Základný typ vrcholu má konštruktor s dvomi parametrami, prvý definuje meno vrcholu a druhý určí jeho farbu.

```
var node1 = new DefaultNodeModel('Node 1', 'rgb(0,192,255)');
let port = node1.addOutPort('Out');
let port3 = node1.addInPort('In');
node1.setPosition(100, 100);
```

Kód 2.19: Vytvorenie vrcholu

Týmto sme vytvorili vrchol s dvomi špecifickými portami. Na tomto vrchole môžeme vytvárať hrany obomi smermi, teda hrany môžu do tohto vrcholu smerovať, ale aj z neho ísť. Pozíciu vrcholu v plátne nastavíme cez metódu `setPosition`, ktorý má dva parametre, predstavujúce súradnice x a y. Avšak momentálne sa ešte vrchol nezobrazí v plátne. Je nutné ho pridať do vytvoreného modelu. Na zobrazenie vytvoreného vrcholu v plátne v danom modeli slúži funkcia `addAll`.

```
activeModel.addAll(node1);
```

Kód 2.20: Pridanie vrcholu do modelu

Táto funkcia je špecifická pre daný model, keďže diagram môže mať viacero modelov, každý predstavujúci rozdielny diagram. Medzi týmito modelmi je následne možné prepínať menením aktívneho modelu diagram enginu. Na to slúži funkcia `setModel`.

Čo sa týka samotných objektov grafu, tak tieto sa dajú upravovať a meniť. Každý objekt grafu sa skladá z viacerých častí. Má svoj model, widget a factory.

Model obsahuje stav objektu a funkcie na vykonávanie zmien v objekte. Stav predstavuje atribúty daného objektu, ako je meno, pozícia, či je objekt zablokovaný (teda, že sa s ním nedá pohybovať v grafe a nedá sa ani zmazať), šírku, výšku, farbu a pod. Funkcie slúžia zároveň na určovanie vlastnostní objektu, napríklad v prípade portu vieme pridať kontrolu, že kedy je možné daný port prepojiť linkou, resp. koľko liniek môže do daného portu ísť. V prípade vrcholu, model obsahuje ešte pole portov a funkcie na ich pridávanie a odoberanie.

```
export class NodeModel<G extends NodeModelGenerics = NodeModelGenerics> extends
BasePositionModel<G> {
    protected ports: { [s: string]: PortModel };

    constructor(options: G['OPTIONS']) {
        super(options);
        this.ports = {};
    }

    getPorts(): { [s: string]: PortModel } {
```

```
        return this.ports;
    }
}
```

Kód 2.21: Ukážka časti modelu vrcholu

Widget slúži na definovanie vzhľadu objektu. Ide o React komponent, ktorý berie údaje z modelu a zobrazí ich v grafickej podobe. Vzhľad je možné ľubovoľne meniť. Knižnica využíva `styled.div`, ale je možné vytvoriť aj vlastné plnohodnotné komponenty.

```
export const Node = styled.div<{ background: string; selected: boolean }>`
 background-color: ${p => p.background};
 font-family: sans-serif;
 overflow: visible;
 font-size: 11px;
 border: solid 2px ${p => (p.selected ? 'rgb(0,192,255)' : 'black')};
`;
```

Kód 2.22: Ukážka styled.div komponentu

Widget väčšiny objektov grafu je tvorený viacerými (menšími) komponentmi ako je v ukážke kódu 2.23. Aby bola práca s objektami interaktívna, tak widget reaguje na zmenu stavu modelu. Teda ak sa niečo zmení v modeli, tak sa objekt prekreslí. Avšak v základnej verzii sa objekty prekresľujú iba v prípade už zadaných premenných modelu. Ak si programátor dodefinuje nové premenné, tak pri ich zmene sa bez dodatočných zmien v kóde graf nebude aktualizovať. Na aktualizovanie grafu je nutné zavolať funkciu `repaintCanvas`. Táto funkcia je súčasťou diagram enginu, teda je potrebné, aby mal widget referenciu na engine objekt. Widget teda funguje ako klasický React komponent, trieda môže mať svoj vlastný state, môžeme pridávať udalosti a preposielať dáta medzi jednotlivými komponentami ako props.

```
render() {
    return (
        <Node selected={this.props.model.isSelected()}
              background={this.props.model.getOptions().color}>
            <Title>
                <TitleName>{this.props.model.getOptions().name}</TitleName>
            </Title>
            <Ports>
                <PortsContainer>{_.map(this.props.model.getPorts(),
this.generatePort)}</PortsContainer>
            </Ports>
        </Node>
    );
}
```

Kód 2.23: Ukážka `render` metódy v rámci widget komponentu

Factory slúži na prepojenie modelu a widgetu. Pri používaní vlastných typov modelov je zároveň nutné tieto nové typy zaregistrovať v rámci enginu, aby ich bolo možné vytvárať. Na to slúži funkcia `registerFactory`, ktorá berie ako parameter factory pre príslušný (nový) typ.

2.6.2 React Digraph

React Digraph [14] je knižnica, ktorá je primárne robená na vytváranie orientovaných grafov. Tento grafový editor je tvorený pomocou Reactu a jednotlivé časti grafu predstavujú React komponenty.

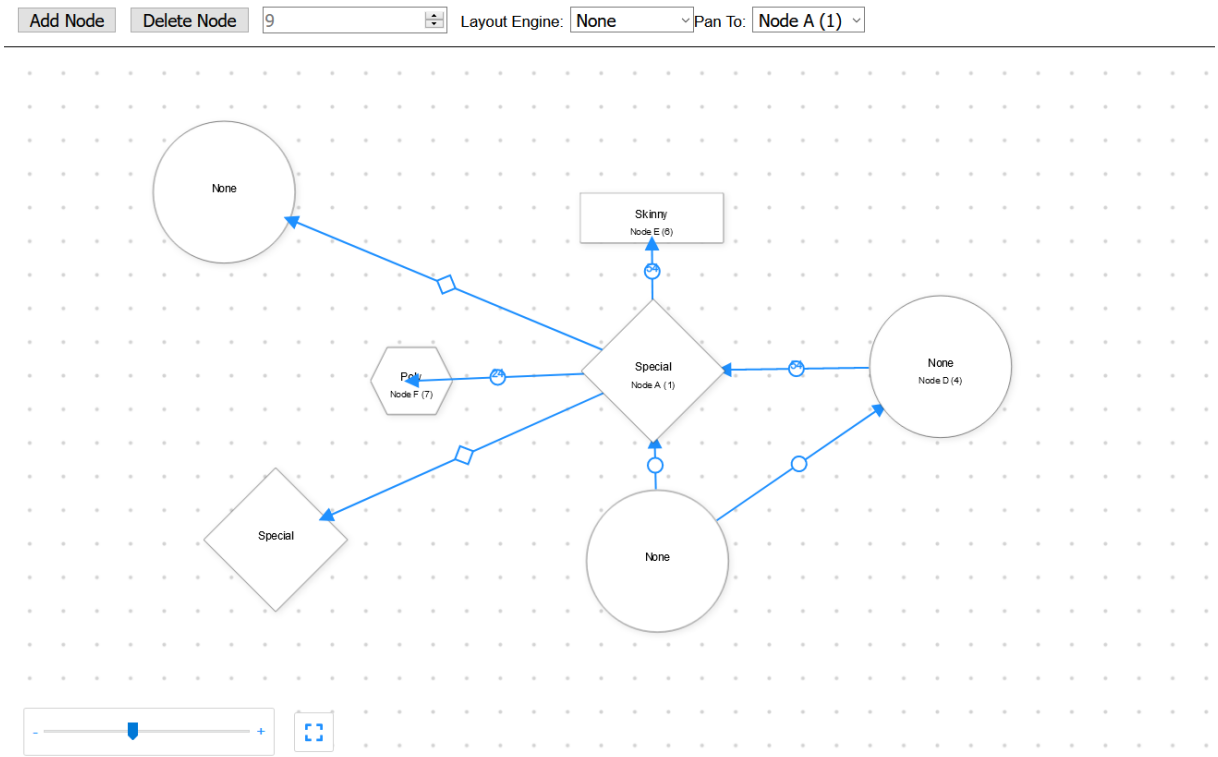
Graf sa skladá z vrcholov a hrán. Vrcholy je možné pridávať, mazať, presúvať a pomenovať. Vrcholy majú vo svojich atribútoch (properties) definovanú veľkosť, farbu a tvar. Udalosti nad plochou povoľujú jej ťahania, zatiaľ čo ostatné udalosti slúžia na prácu s vrcholmi a hranami.

Orientovanou hranou spájame jednotlivé vrcholy. V základnej verzii sa pri vytváraní vrcholu vyberie náhodne jeden z dvoch tvarov (buď kruh alebo štvorec). Hrana je orientovaná a smeruje z vrcholu, z ktorého bola vytvorená do druhého vrcholu. Hrana predstavuje prepojenie medzi dvomi vrcholmi, inak ju nejde vytvoriť. Ak vytvoríme hranu bez koncového vrcholu (koncový vrchol je taký vrchol, kde má hrana smerovať), tak takáto hrana sa vymaže. Medzi každou unikátnou dvojicou vrcholov môže existovať iba jedna hrana (nejde ju duplikovať). Jednotlivé vrcholy a hrany sú uložené v stave pre graf.

```
"nodes": [
    {
      "id": 1,
      "title": "Node A",
      "x": 258.3976135253906,
      "y": 331.9783248901367,
      "type": "empty"
    }]
```

Kód 2.24: Stav vrcholu

Časti editora sa dajú prispôsobiť podľa potrieb programátora. Knižnica podporuje vytváranie vlastných typov vrcholov, v základnej verzií podporuje automatické rozloženie grafu a poskytuje množstvo klávesových skratiek, ktorými je možné graf ovládať. Okrem toho graf obsahuje funkciu, ktorá pridáva možnosť zamerať sa na určitý vrchol (focus). To je veľkou výhodou hlavne pri väčších grafoch, kde môže byť hľadanie konkrétneho vrcholu zložitejšie. Základný layout grafu obsahuje tlačidlá na pridanie, mazanie vrcholu, rozloženie grafu, focus na určitý vrchol a zoom tlačidlá. Zoom tlačidlá dovoľujú graf priblížiť alebo oddialiť a okrem toho je možné ešte pomocou druhého zoom tlačidla nastaviť plátno tak, aby bolo vidieť úplne celý graf. Dokumentácia je zrozumiteľná a jednotlivé objekty a funkcie sú detailne opísané, teda pochopenie knižnice je jednoduchšie. Oproti React Diagrams ide ale o menší projekt s menšou komunitou.



Obr. 2.7: Grafový editor React Digraph, spolu s vrcholmi a linkami

3 Podobné práce

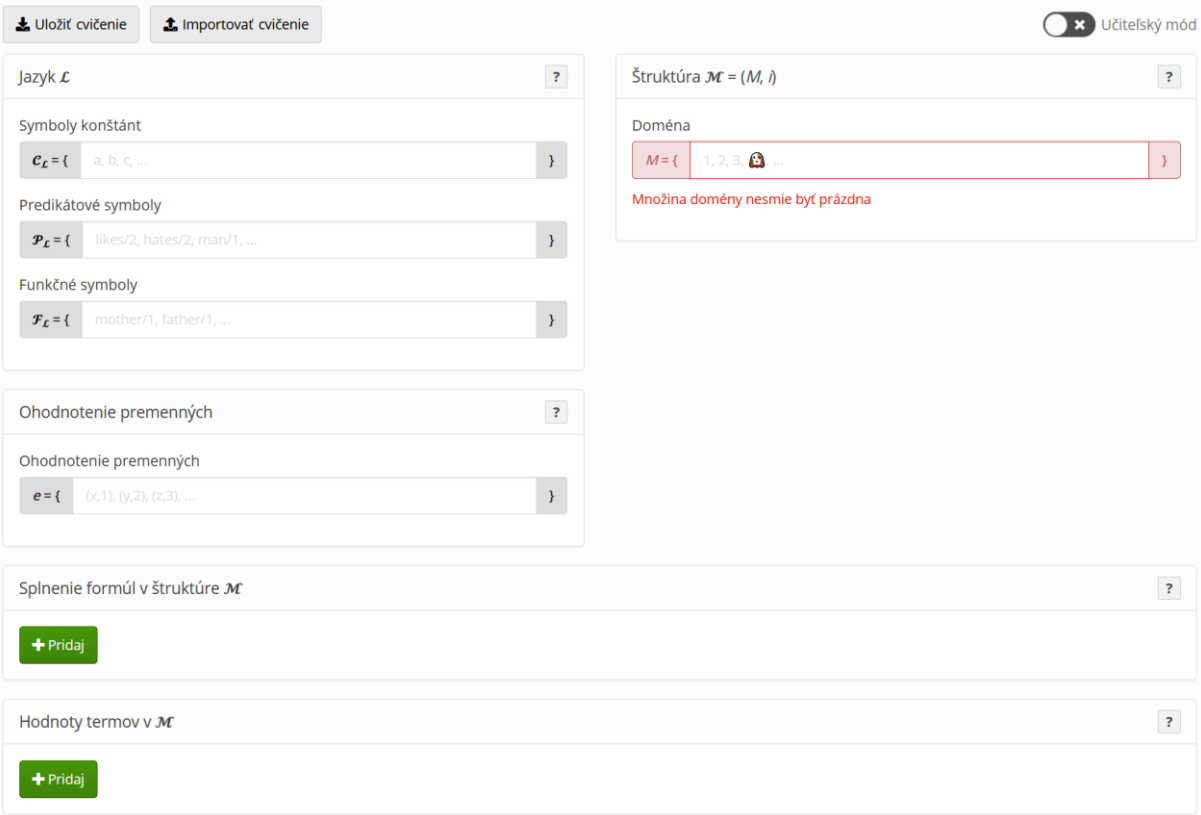
V tejto kapitole spomeniem podobné práce. Moja bakalárska práca priamo nadväzuje na prácu uvedenú v sekcii 3.1, ktorej kód využívam a modifikujem. Okrem toho boli tieto práce preštudované za účelom získania prehľadu o rôznych technológiách.

3.1 Prieskumník sémantiky logiky prvého rádu

Prieskumník sémantiky logiky prvého rádu [1] je bakalárska práca Milana Cifru. Aplikácia[1] predstavuje výučbový nástroj, ktorý je primárne určený pre študentov predmetu Matematika (4) – Logika pre informatikov.

Študenti si v aplikácii môžu precvičovať prácu so štruktúrami, teda vedia si zadefinovať prvky domény, určiť symboly konštánt, predikátové a funkčné symboly, priraďovať týmto mimologickým symbolom množiny prvkov alebo *n*-tíc prvkov z domény cez interpretačnú funkciu a nakoniec pridávať a vyhodnocovať formuly a termy.

Aplikácia beží na strane klienta. Používateľské rozhranie je vytvorené pomocou frameworku React a na uchovávanie stavu aplikácie sa využíva knižnica Redux. Na spracovanie vstupného textu od používateľa slúži syntaktický analyzátor (parser) PEG.js, pomocou ktorého sú vytvorené pravidlá, ktoré text musí spĺňať, aby bol akceptovaný.



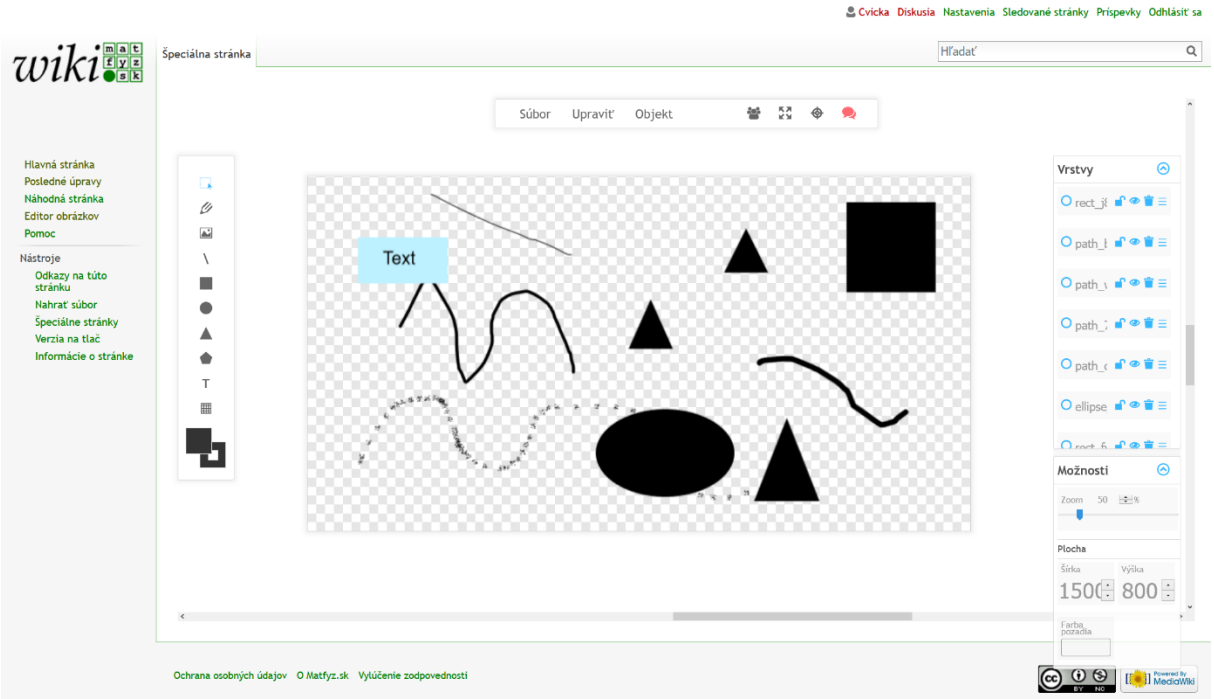
Obr. 3.1: Náhľad finálnej verzie aplikácie Prieskumník sémantiky logiky prvého rádu

[1] https://github.com/FMFI-UK-1-AIN-412/structure-explorer/

3.2 Kolaboratívny grafický editor pre MediaWiki

Kolaboratívny grafický editor pre MediaWiki [15] predstavuje diplomovú prácu Martina Krasňana. Aplikácia[2] slúži na vytváranie a menenie obrázkov v rámci MediaWiki systémov.

Časť tejto práce sa venovala vykresľovaniu plochy a prácu s udalosťami. Bližšie v nej bola zanalyzovaná HTML element canvas, slúžiaca na vykresľovanie grafiky na webových stránkach. Okrem toho v práci bola spomenutá knižnica Fabric.JS, ktorá predstavuje nadstavbu pre canvas. Fabric.JS sa stará o udržiavanie stavu canvasu, prekresľovanie grafickej plochy a dovoľuje priamu prácu s jednotlivými objektami plochy. Poslednou technológiou, ktorú mi táto práca priblížila bola Draw.IO. Draw.IO je aplikácia na tvorbu vektorovej grafiky. Je využívaná širokou verejnosť. Zameral som sa hlavne na jej používateľské rozhranie, ktoré je intuitívne a ľahko pochopiteľné aj pre začiatočníkov. Predstavuje pre mňa inšpiráciu ako vytvoriť dobré používateľské rozhranie.



Obr. 3.2: Náhľad finálnej verzie aplikácie Kolaboratívny grafický editor pre MediaWiki implementovaný na stránke wiki.matfyz.sk

[2] https://github.com/krasnan/ImageEditor

4 Prieskumník sémantiky logiky prvého rádu

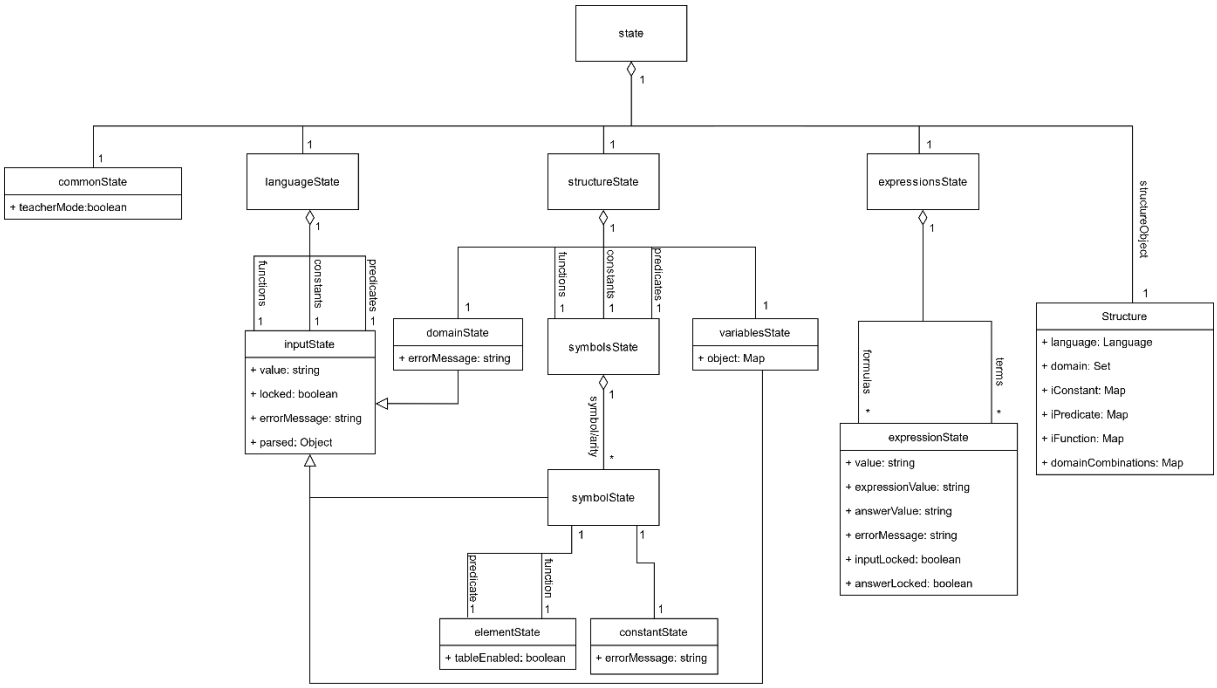
V tejto kapitole presnejšie opíšem fungovanie aplikácie Prieskumník sémantiky logiky prvého rádu, ktorý je stručne opísaný v sekcii 3.1. Keďže mnou vyvíjaná aplikácia priamo nadväzuje na tento prieskumník, dobré pochopenie zdrojového kódu tejto práce je dôležitou súčasťou pri vytváraní mojej aplikácie. Pochopenie kódu mi dovolí pridať nové prvky (ako je databázový a grafový pohľad) a zároveň dobrá znalosť kódu mi dovolí vykonať zmeny už v existujúcom kóde, ako je refaktorizácia tých častí kódu, ktoré sú neprehľadné a duplicitné, alebo aktualizácia knižnice Bootstrap. Zároveň je nutné opísať ako táto aplikácia fungovala a aká bola jej štruktúra predtým ako som začal s ňou pracovať, aby bolo zreteľné, ktoré časti prieskumníka už boli predtým naprogramované.

4.1 Organizácia kódu

V tejto sekcii opíšem štruktúru kódu. Zdrojový kód aplikácie sa nachádza v zložke `/src`. Všetky Reactové komponenty stránky, ktoré tvoria používateľské rozhranie, sú uložené v `components/`. Hlavným komponentom aplikácie je `App.js`, v ktorom je inicializovaný `store`. `Store` je súčasťou knižnice Redux a obsahuje stav aplikácie. Ostatné komponenty sa dostávajú k stavu aplikácie práve prostredníctvom tohto komponentu, pretože sú v ňom zaobalené do komponentu `Provider`. Súbory týkajúce sa spravovania stavu aplikácie sú v zložke `redux/`, pričom Reduxové akcie sú v zložke `redux/actions/`, reducery sa nachádzajú v `redux/ reducers/` a kontajnery sú v `redux/containers/`. Následne je tu zložka `constants/`, ktorá je určená pre konštanty využívané v aplikácii. Priečinok `parser/` obsahuje zdrojové súbory syntaktického analyzátora PEG.js. Zložka `model/` má v sebe zdrojové súbory dátového modelu aplikácie a `tests/` obsahuje testy pre aplikáciu.

Štýl stránky je robený pomocou knižnice Bootstrap, ktorý je ešte následne prispôsobený. Všetky súbory týkajúce sa štýlu sa nachádzajú v zložkách `/public` a v zložke `/src/sass`, ktorej štýl je naprogramovaný v jazyku Sass a následne je skompilovaný pomocou kompilátora Gulp do CSS jazyka. Tento štýl je následne umiestnený do súboru `/public/css/style.css`.

4.2 Štruktúra stavu



Obr. 4.1: Reduxový stav Prieskumníka sémantiky logiky prvého rádu

Stav aplikácie sa skladá z viacerých častí. Sú to `commonState`, `language`, `structure`, `expressions` a `structureObject`.

Atribút `commonState` slúži pre globálne premenné, ktoré sa týkajú celej aplikácie. Atribút `commonState` sa skladá z premennej `teacherMode`, ktorá riadi učiteľský mód. Učiteľský mód slúži na ovládanie uzamykania častí matematického pohľadu (ako sú textové polia pre doménu, predikátové symboly alebo znenie jednotlivých formúl, resp. či má byť formula splnená a pod.).

Väčšina častí stavu pracuje s objektom `inputState` a `expressionState`. Tieto objekty majú spoločné premenné `value`, `locked`, `errorMessage` a `parsed`. Avšak, premenná `parsed` sa v objektoch definuje až pri syntaktickej analýze, nie pri ich vytvorení. Pri vytvorení sa teda premenná `parsed` v objektoch `inputState` a `expressionState` nenachádza. Ďalšou premennou je `value`, ktorej hodnota je text zadaný používateľom. Čiarky v texte premennej `value` rozdeľujú text na menšie časti, teda slúžia ako oddeľovač. Toto rozdelenie je vykonané pri syntaktickej analýze. Premenná `locked` je typu `boolean` a slúži na samotné uzamykanie jednotlivých vstupov. Premenná `errorMessage` obsahuje textovú správu pre používateľa, ktorá sa vypíše v prípade chyby v textovom poli (teda keď sa nepodarí v danom poli vykonať syntaktická analýza). Poslednou premennou je `parsed`, ktorej presná definícia (obsah) sa líši podľa jednotlivých častí stavu.

Atribút `language` obsahuje symboly konštánt, predikátové a funkčné symboly. Premenná `parsed` je typu `Object`, pričom každá takáto premenná obsahuje atribúty `name` a `arity`. `Name` predstavuje názov symbolu a `arity` je arita symbolu.

Atribút `structure` sa stará o interpretácie konštantných, predikátových a funkčných symbolov a zároveň obsahuje prvky domény a ohodnotenie premenných. Premenná `parsed` je v tomto prípade typu `Array` a jej hodnotou je syntakticky analyzovaný text od používateľa. Po vykonaní syntaktickej analýzy je text rozdelení do polí, pričom každé pole predstavuje časť syntakticky analyzovaného textu. Každé takéto (menšie) pole predstavuje časť interpretačnej funkcie, resp. prvok domény alebo ohodnotenie premennej. V prípade ak text neprejde syntaktickou analýzou, tak hodnota premennej `parsed` je `null`.

Atribút `expressions` spravuje formuly a termy. Premenná `answerLocked` je typu `boolean` a určuje, či je hodnota z ponuky odomknutá alebo zamknutá (teda, že ju nejde meniť). Premenná `inputLocked` je toho istého typu, ale stará sa o prístup k textovému poľu výrazu (teda, že či je textové pole zablokované). Nasledujúca premenná je `answerValue` typu `string`. V prípade ak výraz je typu formula, tak táto premenná nadobúda jednu z troch možných hodnôt: buď `1`, ktorá značí, že formula má byť splnená alebo `-1` pričom formula nemá byť splnená alebo `0`, kedy nie je určené, či má byť formula splnená alebo nesplnená. V prípade ak výraz je typu term, tak hodnota premennej `answerValue` je vybratý prvok z ponuky (ktorý predstavuje prvok domény). Premenná `parsed` sa skladá z viacerých objektov (typ `Object`), pričom každý z týchto objektov je z triedy `Term` alebo `Formula`.

Atribút `structureObject` predstavuje syntakticky analyzované údaje získané z `language` a zo `structure`. Reducery a akcie musia byť naprogramované tak, aby údaje zo `structure`, `language` a `structureObject` boli vždy synchronizované a konzistentné. Atribút expressions sa vyhodnocuje použitím údajov zo `structureObject`, ktoré predstavujú posledné dobré syntakticky analyzované údaje.

4.2.1 Redux

O stav aplikácie sa stará knižnica Redux. Hlavný reducer sa nachádza v root.js, ktorý spravuje ostatné reducery. Ostatné reducery sú pomenované rovnako ako jednotlivé časti stavu, teda `common`, `language`, `structure` a `expressions` reducer. Časť stavu, `structureObject`, nemá samotný reducer, pretože táto časť sa synchronizuje prostredníctvom `language` a `structure` reducerov. Niektoré Redux akcie sa vykonávajú zarovno vo viacerých reduceroch, pretože údaje medzi jednotlivými časťami stavu musia byť synchronizované a zároveň musia ostať konzistentné. Ak sa teda niečo zmení v časti `language`, tak je nutné skontrolovať `structure` a `expressions`. Pri zmene interpretácie symbolu alebo ohodnotenia premennej stačí skontrolovať iba `expressions`. V prípade zmeny v doméne je nutné kontrolovať interpretácie v `structure` a skontrolovať `expressions`.

5 Analýza problému

V tejto kapitole opíšem akú funkcionalitu má mať výsledná aplikácia.

5.1 Zámer aplikácie

Hlavným cieľom mojej aplikácie je poskytnúť online webový nástroj pre študentov predmetu Matematika 4 – Logika pre informatikov. Študentom má pomôcť lepšie pochopiť prácu so štruktúrami v logike prvého rádu.

Aplikácia sa bude skladať z matematickej a grafovej časti. Matematická časť bude predstavovať časť aplikácie, ktorá už je naprogramovaná (bližšie vysvetlená v sekcii 3.1 a kapitole 4). Grafová časť aplikácie bude pridaná a spojená s matematickou časťou. Moja aplikácia by mala prostredníctvom grafu poskytovať podobnú funkcionalitu ako má matematická časť (viď sekcia 3.1). Mala by byť zabezpečená synchronizácia údajov medzi grafovou a matematickou časťou aplikácie. Okrem toho bude kód pôvodnej aplikácie refaktorizovaný, čím bude kód prehľadnejší a práca s ním bude jednoduchšia.

Graf bude obsahovať vrcholy a hrany. V grafe bude existovať viacero typov vrcholov, pričom každý bude mať inú funkcionalitu. Jeden z typov sa bude nazývať doménový vrchol. Spájaním doménových vrcholov sa budú vytvárať hrany. Takéto hrany by mali byť orientované, teda záleží, v ktorom vrchole začína počiatočný bod hrany. Hrany medzi doménovými vrcholmi predstavujú binárne vzťahy (teda predikátové symboly arity dva a funkčné symboly arity jedna). Okrem toho by malo byť možné spájať doménové vrcholy s vrcholmi, ktoré budú reprezentovať konštanty. V takomto prípade nezáleží, či je hrana orientovaná. Pre aritu tri a štyri budú vytvorené nové typy vrcholov, ktoré znázorňujú ternárny a kvaternárny vzťah. Tieto vrcholy by sa mali hranami spájať s doménovými vrcholmi, ktoré predstavujú účastníkov ternárneho, resp. kvaternárneho vzťahu.

5.2 Funkčné požiadavky na aplikáciu

V tejto sekcii priblížim akú funkcionalitu bude mať aplikácia.

1) Pohľady aplikácie
   a) Grafový pohľad má byť implementovaný do pôvodnej aplikácie.
   b) Aplikácia má byť naprogramovaná tak, aby bolo možné prepínať medzi matematickým a grafovým pohľadom.

2) Režimy grafu
   a) Graf má obsahovať režim, ktorý slúži výlučne na pohyb po plátne a na mazanie vrcholov. V takomto móde nebude možné meniť vlastnosti vrcholov.
   b) Graf má obsahovať režim, ktorý slúži na editovanie objektov grafu. V tomto režime bude možné meniť vlastnosti vrcholov. Vlastnosti vrcholov sú špecifické pre každý typ vrcholu. Okrem toho bude v tomto režime možné označiť objekt grafu a bude možné objekty mazať.

3) Panel grafu
   a) Panel grafu bude obsahovať typy vrcholov.
   b) Ťahaním vrcholu z panelu grafu do plátna grafu sa vrchol umiestni do plátna, ktorého pozícia zodpovedá pozícii kurzora. Pokiaľ sa kurzor myši nachádza mimo plátna, tak takýto vrchol sa nevytvorí.

c) Kliknutím na typ vrcholu z panela grafu sa v grafe vytvorí nový vrchol rovnakého typu. Pozícia takéhoto vrcholu v grafe je síce náhodná, ale je ohraničená rozmermi grafu.

4) Vrcholy grafu

Graf sa bude skladať z viacerých typov vrcholov, pričom každý typ bude mať inú funkcionalitu. Aplikácia bude obsahovať tri základné typy vrcholov: doménový vrchol, vrchol konštanty a *n*-árne vrcholy, ktorých arita je tri a štyri.

a) Doménový vrchol

I. Doménový vrchol bude predstavovať prvok domény. Pridanie vrcholu do grafu zároveň pridá prvok domény do matematického pohľadu. Názov doménového prvku v matematickom pohľade zodpovedá názvu vytvoreného doménového vrcholu.

II. Doménový vrchol obsahuje názov. Názov vrcholu je prvkom domény. Názov vrcholu bude možné zmeniť. Pri zmene názvu vrcholu sa zmení aj zodpovedajúci prvok domény. Opačným smerom to nebude fungovať, teda zmena názvu prvku domény v matematickom pohľade nezmení názov vrcholu v grafe, ale zmaže pôvodný vrchol a nahradí ho novým.

III. Doménový vrchol bude obsahovať zoznam unárnych predikátov, v interpretácií ktorých sa nachádza prvok domény, reprezentovaný týmto vrcholov. Predikáty bude možné pridávať a odoberať.

b) Vrchol konštanty

I. Vrchol konštanty bude predstavovať symbol konštanty v matematickom pohľade.

II. Vrchol konštanty obsahuje názov, ktorý je možné meniť. Funguje rovnako ako názov v doménovom vrchole. Názvy vrcholov budú unikátne pre daný typ, teda bude môcť existovať doménový vrchol s rovnakým názvom ako má vrchol konštanty. V rámci jedného typu vrcholu to nebude možné.

c) *N*-árne vrcholy

I. *N*-árny vrchol bude reprezentovať ternárny alebo kvaternárny vzťah pre predikátové a funkčné symboly.

II. Ternárny vrchol bude obsahovať zoznam ternárnych predikátov a binárnych funkcií, v interpretácii ktorých sa nachádza trojica prvkov domény, reprezentovaná týmto vrcholom.

III. Kvaternárny vrchol bude obsahovať zoznam kvaternárnych predikátov a ternárnych funkcií, v interpretácii ktorých sa nachádza štvorica prvkov domény, reprezentovaná týmto vrcholom.

IV. Predikáty a funkcie bude možné pridávať a mazať.

5) Hrany

Správanie hrán závisí od typu počiatočného a konečného vrcholu.

V prípade ak počiatočný alebo konečný vrchol je konštanta a druhý vrchol je doménový prvok, tak vtedy sa daný vrchol pripojí ku konštante. Ak je ku konštante pripojený iný typ vrcholu ako doménový vrchol, tak takáto hrana je neplatná a je zmazaná. Ak medzi vrcholom konštanty a nejakým doménovým vrcholom už existuje

hrana a medzi touto konštantou a doménovým vrcholom vytvoríme novú hranu, pričom tento doménový vrchol je odlišný ako ten v predchádzajúcej hrane, tak pôvodná hrana je zmazaná.

Na hrane sa vytvorí binárny vzťah v prípade, keď počiatočný aj konečný vrchol je doménového typu. Okrem toho je možné vytvoriť binárny vzťah aj v rámci rovnakého doménového vrcholu.

Ak jeden z vrcholov je doménový vrchol a druhý vrchol je *n*-árny, tak vtedy sa vytvorí hrana medzi doménovým vrcholom a uzlovým miestom, ktorý predstavuje parameter v *n*-árnom vrchole. Pokiaľ uzlové miesto *n*-árneho vrcholu bude mať v sebe napojený iný doménový vrchol ako momentálne vytvorená hrana, tak pôvodná hrana sa vymaže.

V iných prípadoch nebude možné vytvoriť hrany a ak sa o to používateľ pokúsi, tak hrana bude zmazaná.

6) Binárne vzťahy
   a) Binárny vzťah bude vytvorený medzi dvomi doménovými vrcholmi alebo v rámci jedného doménového vrcholu.
   b) Binárny vzťah bude obsahovať zoznam binárnych predikátových a unárnych funkčných symbolov. Interpretácie týchto symbolov obsahujú niektorú usporiadanú dvojicu vrcholov, ktoré sú v tomto binárnom vzťahu.
   c) Predikátové a funkčné symboly binárneho vzťahu bude možné pridávať a mazať.

7) Synchronizácia medzi matematickým a grafovým pohľadom
   Každá zmena, ktorá reálne ovplyvňuje matematický pohľad sa synchronizuje s grafovým pohľadom. Synchronizácia sa deje v momente po vykonaní akcie v grafe. Zároveň všetky zmeny vykonané v matematickom pohľade sa musia prejaviť v grafe, ale prejavia sa až po prepnutí matematického pohľadu na grafový (teda nie hneď po vykonaní akcie v matematike). Týmto je zaručený lepší výkon aplikácie, pretože sa graf neustále nemení, keď sme v matematickom pohľade. Okamžitá synchronizácia matematického pohľadu je nutná kvôli vyhodnocovaniu formúl a termov.

   Nasledovné zmeny v grafe vyvolajú synchronizáciu s matematickým pohľadom:
   a) Vytvorenie doménového vrcholu alebo vrchol konštanty.
   b) Premenovanie vrcholu.
   c) Vymazanie doménového vrcholu alebo vrcholu konštanty.
   d) Vymazanie *n*-árneho vrcholu, ak žiaden z portov vrcholu nie je prázdny.
   e) Úspešné pridanie predikátu alebo funkcie do vrcholu, ktorý ešte nemá pridávaný symbol v zozname.
   f) Zmazania predikátu alebo funkcie zo zoznamu vrcholu.
   g) Pridanie alebo zmazanie predikátového alebo funkčného symbolu z binárneho vzťahu.
   h) Zmena smeru binárneho symbolu.
   i) Vytvorenie alebo zmazanie hrany medzi doménovým vrcholom a vrcholom konštanty.
   j) Vytvorenie hrany medzi doménovým vrcholom a ternárnym alebo kvaternárnym vrcholom, pričom do všetkých ostatných portov *n*-árneho vrcholu už vedú hrany.

8) Export/Import grafu
   a) V prípade exportovania cvičenia sa súradnice vrcholov grafu musia uložiť do súboru s cvičením.
   b) V prípade exportu cvičenia a následného importu súboru, ktorý obsahoval vrcholy grafu, musia mať vrcholy rovnaké pozície ako v čase exportu.

9) Kontrola validity textových polí
   a) Textové polia grafu sa musia validovať pri každej zmene.
   b) V prípade neplatného textu sa tento označí inou farbou.
   c) Neplatný text nebude akceptovaný pri stlačení tlačidiel vrcholov grafu. Pravidlá validácie sa líšia podľa typu vrcholu.

10) Databázový pohľad
    Matematický pohľad aplikácie má byť vylepšený o funkcionalitu zobrazovania interpretácie predikátových a funkčných symbolov štýlom relačných databáz.

5.3 Ostatné požiadavky na aplikáciu

Do tejto sekcie patria požiadavky, ktoré priamo neovplyvňujú fungovanie jadra aplikácie, ale ktoré bude vhodné implementovať:

1) Vzhľad tlačidiel v paneli grafu
   Vzhľad tlačidiel v paneli grafu má reprezentovať vzhľad vrcholov v grafe.

2) Pomocník k prvkom grafu
   Každý objekt grafu bude obsahovať nápovedu. Nápoveda bude vysvetľovať hlavnú funkcionalitu objektu. Nápoveda sa zobrazí po nadídení myšou ponad konkrétny objekt.
3) Dynamické menenie veľkosti textového poľa
   Textové polia v grafe sa dynamicky zväčšujú alebo zmenšujú v závislosti od počtu znakov v textovom poli.

4) Aktualizácia Bootstrap knižnice
   Pre prepojenie viacerých aplikácií pre predmet Matematika 4 je nutné skonsolidovať štýly jednotlivých aplikácií. Teda je potrebné aktualizovať Bootstrap knižnicu a štýl upraviť tak, aby bol čo najuniverzálnejší.

5) Refaktorizácia existujúceho kódu
   Existujúci kód by bolo dobré refaktorizovať, aby práca s ním bola jednoduchšia a aby sa zároveň odstránili duplicity v kóde.

6 Návrh

6.1 Výber grafového editora

Jednou z možností pri výbere grafového editora bolo vytvoriť si vlastný editor. Po zvážení okolností sme so školiteľom dospeli k záveru, že vytvorenie vlastného editora by bolo časovo náročné. Rozhodli sme sa teda použiť už existujúcu grafovú knižnicu.

Editory musia spĺňať vopred definované podmienky. Editor musí byť naprogramovaný pomocou JavaScriptovej knižnice React. Zároveň musí ísť o voľne dostupný (neplatený) program. Zároveň musí podporovať vytváranie vlastných typov objektov grafu, keďže v našom prípade vyvíjame aplikáciu so špecifickým zameraním.

Finálny výber sa zúžil na editor *React Diagrams* a *React Digraph*. Obidva editory zvládajú bezproblémovo formátovanie textu, čo predstavuje zásadnú výhodu, nakoľko v grafe sa pracuje aj s textom. Vybrali sme nakoniec editor *React Diagrams*. Tento editor je komplexnejší a dovoľuje nám na ňom ľahšie stavať. Zároveň uvedený graf je intuitívnejší a umožňuje napríklad označiť viacero vrcholov naraz. Okrem toho bol pre náš konkrétny cieľ lepšie zameraný, nakoľko základná verzia editora bola oveľa príjemnejšia na používanie. Základné typy objektov grafu vieme zároveň po určitých zmenách použiť. Obdobne, komunita stojaca za knižnicou React Diagrams je oveľa rozsiahlejšia a teda riešenie chýb a problémov je jednoduchšie.

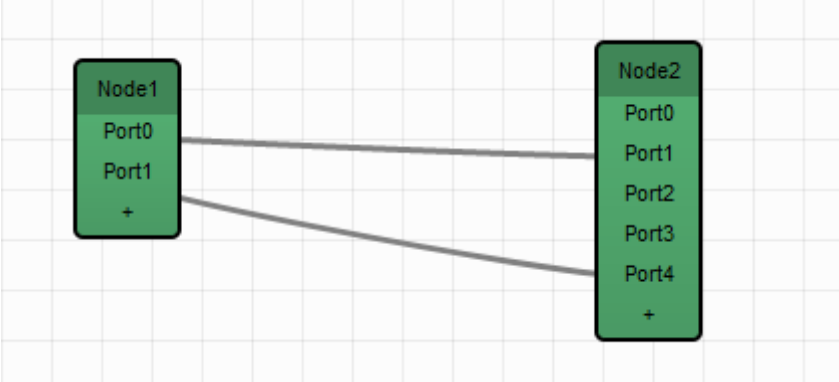
6.2 Vývoj aplikácie

Priebeh vývoja aplikácie nebol lineárny a množstvo častí sa menilo s pribúdajúcou funkcionalitou. V nasledovnej sekcii priblížim, ako prebiehal vývoj aplikácie. Spomeniem tri návrhové etapy a vysvetlím, čo nás viedlo k zmenám v návrhu.

6.2.1 Prvý návrh

Základný typ vrcholu nám úplne nevyhovoval a bolo nutné ho prerobiť. V základnom type vrcholu bolo možné prepojiť jednotlivé porty medzi sebou (aj v rámci seba, ale taktiež prepojiť in port s iným in portom). Okrem toho bolo možné vytvárať uzly, ktoré nikam nevedú (teda koncový port hrany je prázdny – predstavujúci bod na hrane). Klikanie na hranu nespôsobilo označenie hrany, ale vytváralo na nej body. Takáto vlastnosť bola dosť mätúca.

V prvom návrhu som odstránil in and out porty a nahradil ich univerzálnym portom. Existoval teda iba jeden typ portu, ktorý mohol tvoriť hranu s hociktorým iným portom, okrem portov, ktoré boli v rámci toho istého vrcholu (teda začiatočný a koncový port musí pochádzať z rozdielneho vrcholu). Každý port predstavoval funkčný alebo predikátový symbol. Okrem toho som zabezpečil, aby sa meno portu zobrazovalo (graficky) vo vrchole. Meno predstavovalo komponent (`TitleName`), ktorý tvoril časť widgetu vrcholu. Dvojklikom na tento komponent sa meno zmenilo na textové pole, do ktorého bolo možné zadať nový názov vrcholu. V tejto etape vývoja bol každý binárny symbol (funkčný a predikátový) vyjadrený samostatnou hranou. Každá hrana bola orientovaná, koncový bod hrany mal byť označený šípkou, ktorá mala vyjadrovať smer. Od tohto nápadu sme nakoniec upustili, keďže tento spôsob reprezentovania vzťahov spôsoboval, že v grafe bolo veľmi veľa hrán a tým bol graf neprehľadný. V tejto etape návrhu som zároveň zmenil udalosti na hranách, kde kliknutie na hranu spôsobovalo jej označenie (nie vytváranie bodov).

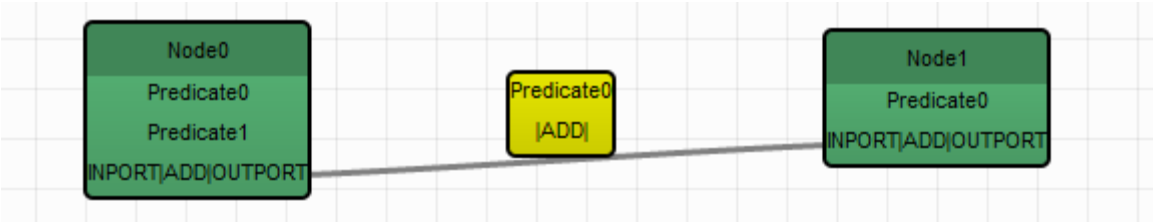


Obr. 6.1: Prvý návrh vrcholov s portami, niektoré z portov sú prepojené (tvoria hranu)

6.2.2 Druhý návrh

V druhom návrhu som sa vrátil k myšlienke in a out portov. In a out porty mali byť primárne určené pre binárne vzťahy a zároveň mali byť nimi vyjadrené smery (z ktorého vrcholu ideme do ktorého). Samotný zoznam symbolov pre binárny vzťah bol reprezentovaný označením (label), ktorý bol umiestnený v strede vytvorenej hrany. V základnej verzii editora označenia mohli obsahovať iba reťazce. Keďže v mojom prípade toto nebolo dostačujúce, tak ich bolo nutné vylepšiť. Vytvoril som teda typ nadpisu, ktorý bol zložený z modelu a widgetu vrcholu, ktorý bol ešte následne zaobalený samotným kódom nadpisu. Kód nadpisu bol takto veľmi zložitý. Tlačidlo ADD malo slúžiť na pridanie nového symbolu do zoznamu. Meno takto vytvoreného symbolu bolo generické a preto by bolo nutné zabezpečiť premenovávanie symbolov. Dvojklikom na názov symbolu sa symbol vymazal zo zoznamu. Okrem toho som v tejto etape návrhu pridal možnosť vytvárať body na hrane pomocou kombinácie klávesnice a myši, čím sa dal graf viacej prispôsobiť.

Oproti predošlému návrhu sa takto graf sprehľadnil, ale toto riešenie nebolo stále postačujúce. Vytváranie hrán obomi smermi spôsobovalo prekrývanie nadpisov. Okrem toho vznikalo v matematike veľa generických symbolov (ako napríklad Predicate0), ktoré sa nikdy nepoužili. Zároveň in a out porty nepomáhali k reprezentácii ternárnych a kvaternárnych vzťahov.



Obr. 6.2: Druhý návrh vrcholov s in a out portami a s hranou vyjadrujúcou binárny vzťah

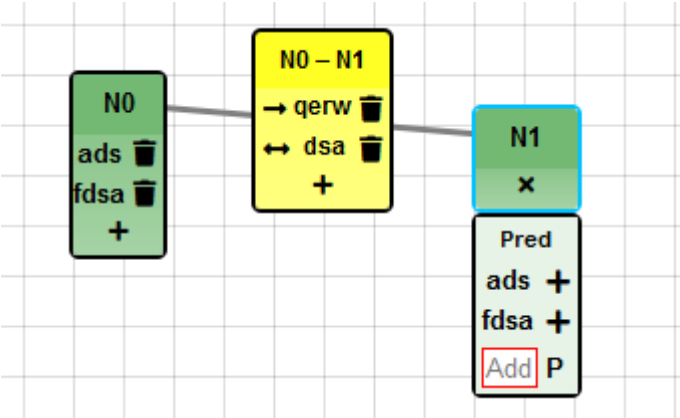
6.2.3 Tretí, finálny návrh

Tretí návrh predstavuje graf vo finálnej verzii aplikácie. Do tejto verzie boli pridané režimy pre graf – editovací a pohybovací. Editovací režim slúžil na prácu s objektami grafu, zatiaľ čo pohybovací režim bol primárne určený na presúvanie objektov v plátne a ich mazanie. Vďaka režimom grafu bolo možné využiť už existujúce časti vrcholov, ktorých funkcionalita sa menila v závislosti od režimu grafu. Jednou z takých častí je meno vrcholu. V editovacom režime slúžil ako port na vytváranie hrán a na menenie názvu vrcholu, zatiaľ čo v pohyblivom režime slúžil ako miesto na ťahanie vrcholu.

Okrem toho bola štruktúra kódu nadpisu prerobená a nadpis dostal nové vlastnosti. Jednou z nich boli smery pre každý symbol v zozname nadpisu, ktorými sa určoval smer binárneho

vzťahu medzi dvomi doménovými vrcholmi iba v rámci jednej hrany (na vyjadrenie vzájomného vzťahu neboli nutné dve hrany). Týmto sa výrazne zmenšil počet hrán v grafe. Taktiež odbudla potreba, aby vrcholy mali in a out porty a preto boli v tomto návrhu úplne odstránené a nahradené jedným typom portov.

Hrany sa zároveň dynamicky menili, teda ich funkcionalita závisela od typu vrcholov, ktoré spájali. Napríklad vytvorenie hrany medzi dvomi doménovými vrcholmi spôsobilo pridanie nadpisu na hrane. V prípade vytvorenia hrany medzi konštantovým a doménovým vrcholom sa nadpis nevytváral. Poslednou veľkou zmenou v tomto návrhu bolo pridanie rozbaľovacej ponuky. Rozbaľovacia ponuka obsahovala textové pole slúžiace na pridávanie symbolov. Toto textové pole nahradilo pridávanie symbolov s generickým názvom. Takto už nebolo nutné symboly premenovávať. Okrem toho obsahovala zoznam použiteľných predikátových a funkčných symbolov, ktoré mohli byť pre daný typ vrcholu použité a dali sa cez tento zoznam ľahko pridávať.



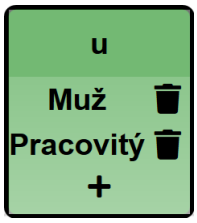
Obr. 6.3: Tretí návrh vrcholov, s hranou vyjadrujúcou binárny vzťah a otvorená rozbaľovacia ponuka vrcholu

6.3 Návrh vzhľadu vrcholov

V tejto sekcii ukážem a stručne vysvetlím návrh vzhľadu jednotlivých vrcholov. Aplikácia bude zahŕňať štyri typy vrcholov a špeciálny vzťah na hrane. Ide o doménový vrchol, vrchol konštanty, ternárny a kvaternárny vrchol. Posledným typom je binárny vzťah.

6.3.1 Doménový vrchol

Doménový vrchol slúži na reprezentáciu prvkov domény. Skladá sa z mena vrcholu (teda prvku domény), zoznamu unárnych predikátov, tlačidla pre rozbaľovaciu ponuku a z rozbaľovacej ponuky.



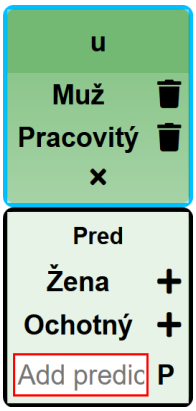
Obr. 6.4 Doménový vrchol obsahujúci unárne predikáty

Meno vrcholu predstavuje prvku domény. Nachádza sa vo vrchnej časti vrcholu. Pod menom vrcholu sa nachádza zoznam unárnych predikátov. Pri každom unárnom predikáte je tlačidlo s ikonou, ktoré slúži na zmazanie predikátu zo zoznamu vrcholu. Za zoznamom

nasleduje tlačidlo s ikonou +, ktorým sa otvára, resp. zatvára rozbaľovacia ponuka. Keď je otvorená rozbaľovacia ponuka, tak ikona tohto tlačidla sa zmení na krížik.

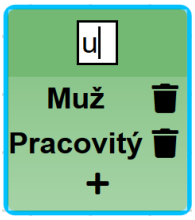
V rozbaľovacej ponuke sa nachádzajú použiteľné predikáty. Pri každom predikáte sa nachádza tlačidlo s ikonou +, ktoré slúži na pridanie predikátu do zoznamu vrcholu. Zoznam v rozbaľovacej ponuke môže byť aj prázdny, pokiaľ nie je definovaný žiaden unárny predikát alebo všetky dostupné unárne predikáty sú už v zozname predikátov samotného vrcholu.

Okrem toho rozbaľovacia ponuka obsahuje textové pole, do ktorého je možné napísať názov pre nový predikát a pridať ho do zoznamu. Vedľa textového poľa sa nachádza tlačidlo, ktorým je možné nový predikát pridať. Pokiaľ je predikát možné pridať, tak okolie tlačidla je po nadídení myšou ofarbené na zeleno. Tlačidlo ostane nezmenené, keď je text v textovom poli nesprávny. Ďalej, pokiaľ je už predikát s daným názvom použitý pre inú aritu alebo pre iný typ symbolu, tak takýto symbol nejde pridať do zoznamu vrcholu. V takom prípade okraje textového poľa nie sú zvýraznené na červeno, pretože text je správny, ale tlačidlo na pridanie bude deaktivované a text v ňom má červenú farbu.



Obr. 6.5 Doménový vrchol s otvorenou rozbaľovaciou ponukou

V editovacom režime je možné meniť viaceré vlastnosti vrcholu. Dvojklikom na meno vrcholu sa daná časť vrcholu zmení na textové pole, do ktorého je možné zadať nový názov vrcholu. Pokiaľ toto textové pole obsahuje nepovolené znaky alebo je toto pole prázdne, tak sa okraje textového poľa vyfarbia na červeno. Okrem toho ťahaním mena vrcholu vytvárame hranu.



Obr. 6.6 Doménový vrchol pri premenovávaní

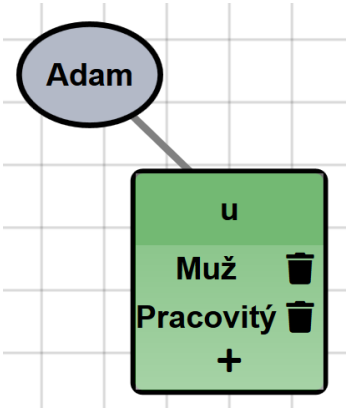
6.3.2 Vrchol konštanty

Vrchol konštanty slúži na interpretovanie konštánt. Skladá sa z mena vrcholu.

Obr. 6.7 Zobrazenie vrcholu konštanty v grafe

Meno vrcholu sa v editovacom režime mení na port, ktorým je možné ťahať a vytvárať tak hrany s doménovými vrcholmi. Vytvorenie takejto hrany reprezentuje v matematike priradenie

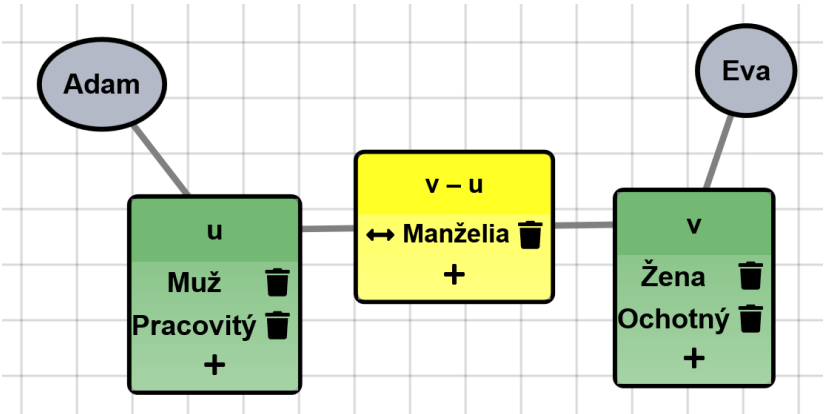
prvku domény (doménového vrcholu) konštante (konštantový vrchol). S ostatnými typmi vrcholov sa nedajú vytvárať hrany. Ak sa o to používateľ pokúsi, tak takáto hrana bude zmazaná. Pokiaľ už do konštanty viedla nejaká hrana a používateľ do tejto konštanty natiahne novú hranu z iného doménového vrcholu, tak pôvodná hrana bude zmazaná. Okrem toho, dvojklikom na meno vrcholu je možné zmeniť meno konštanty.



Obr. 6.8 Hrana medzi vrcholom konštanty a doménovým vrcholom

6.3.3 Binárny vzťah

Binárne vzťahy slúžia na reprezentáciu binárnych predikátov a unárnych funkcií. Binárne vzťahy sú vytvorené na hranách medzi doménovými vrcholmi. Prvá časť vrcholu predstavuje tlačidlo na určenie smeru hrany (z ktorého vrcholu hrana smeruje, do ktorého vrcholu). Stlačením tohto tlačidla sa prepne smer, pričom existujú tri smery hrany. Prvý smer ide z počiatočného vrcholu do konečného, druhy z konečného do počiatočného a tretí smer ide v oboch smeroch. Za tým nasleduje zoznam predikátových a funkčných symbolov. Každý symbol v zozname má na ľavej strane tlačidlo, ktorým sa prepínajú smery. Na pravej strane sa nachádza tlačidlo na zmazanie symbolu. Poslednou časťou vrcholu je tlačidlo na otvorenie alebo zavretie rozbaľovacej ponuky. Rozbaľovacia ponuka funguje podobne ako rozbaľovacia ponuka doménových vrcholov s tým rozdielom, že obsahuje binárne predikáty a unárne funkčné symboly.

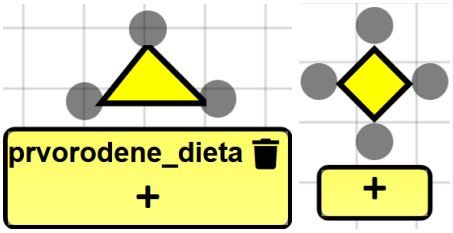


Obr. 6.9 Vytvorená hrana medzi prvkami domény, tvoriaci binárny vzťah

6.3.4 Ternárny a kvaternárny vrchol

Ternárny a kvaternárny vrchol slúžia na zobrazenie ternárnych a kvaternárnych vzťahov. Ternárny vrchol slúži na reprezentáciu ternárnych predikátov a binárnych funkcií. Vrchol sa skladá z tela, ktorý predstavuje trojuholník a tri porty. Kvaternárny vrchol slúži na reprezentáciu kvaternárnych predikátov a ternárnych funkcií. Obsahuje telo, ktoré predstavuje diamant a štyri porty. Porty predstavujú parametre, do ktorých smerujú hrany z doménových

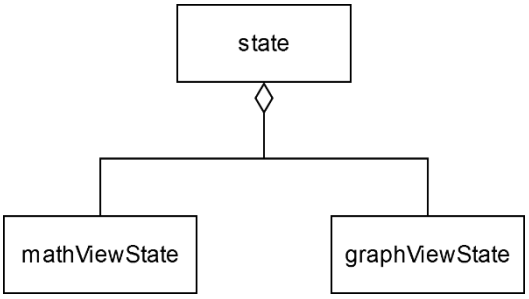
prvkov. Poradie parametrov je dôležité. Každý n-árny vrchol obsahuje zoznam predikátov a funkcií a tlačidlo na otvorenie alebo zavretie rozbaľovacej ponuky.



Obr. 6.10 Ternárny vrchol s binárnou funkciou (vľavo) a kvaternárny vrchol (vpravo)

6.4 Dátový model

Graf si svoj stav v aplikácii uchováva samostatne. Samotný model diagramu obsahuje iba referencie na objekty grafu. Každý objekt grafu si samostatne udržuje svoj vlastný vnútorný stav. V mojej aplikácii využívam dátový model grafu, do ktorého som pridal indexovacie atribúty. Tieto atribúty zrýchľujú prístup k objektom grafu. Indexujem názvami vrcholov, ktoré sú unikátne pre každý typ vrcholu. Nakoniec zjednocujem obidva stavy (matematický a grafový) a vytváram jeden komplexný stav. Každá časť stavu sa vyhodnocuje oddelene, ale je nutné, aby sa stavy medzi sebou synchronizovali. Každý objekt grafu má pevne dané pravidlá a úlohu a reflektuje zmeny v matematickom pohľade. Toto isté platí aj opačne.



Obr. 6.11 Dátový model aplikácie

6.5 Používateľské rozhranie

V tejto sekcii priblížim rozloženie a základnú funkcionalitu komponentov používateľského rozhrania. Rozloženie komponentov je zobrazené na obr. 6.12. Používateľské rozhranie sa skladá zo 7 komponentov:

Navigačný panel: V tomto komponente sa nachádza navigačný panel celej aplikácie. Tento komponent obsahuje tlačidlá na prepínanie sa medzi matematickým a grafovým pohľadom. Okrem toho sú tu tlačidlá na export a import momentálne stavu. Posledné tlačidlo predstavuje pomoc s grafom a zobrazí sa v prípade, iba ak má používateľ práve vybratý grafový pohľad.

Panel grafu: Komponent, ktorý slúži na prácu s grafom, s jeho vlastnosťami a objektami. Obsahuje v sebe komponenty: režim grafu, paleta vrcholov grafu a aj samotný graf. Ide o komponent, ktorý reaguje na klávesové skratky aplikácie (pokiaľ má focus).

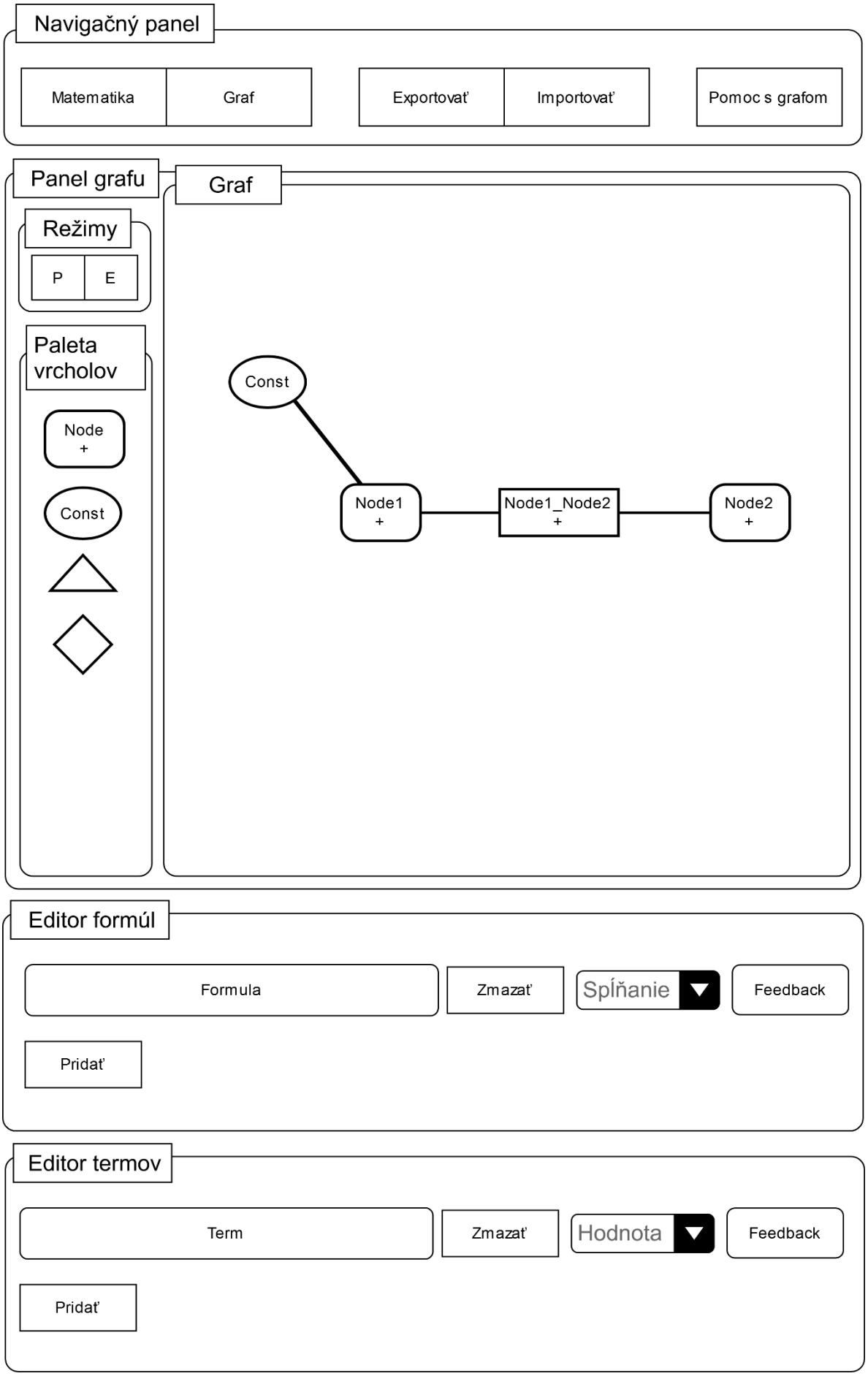
Režim grafu: Tento komponent obsahuje tlačidlá, ktoré slúžia na prepínanie režimov v grafe. Prvé tlačidlo slúži na aktivovanie pohybovacieho režimu v grafe. Druhé tlačidlo slúži na prepnutie sa do editovacieho režimu. Medzi týmito tlačidlami je možné prepínať pomocou klávesovej skratky ;, resp. využiť tlačidlá P a E, kde P aktivuje pohybovací režim a E aktivuje editovací režim. Komponent má definovanú statickú veľkosť a nie je závislý od rozlíšenia prehliadača.

Paleta vrcholov: Komponent, ktorý slúži na pridávanie nových vrcholov do grafu. Tento komponent reaguje na klikanie a ťahanie. Ťahaním vrcholu do plátna grafu sa v grafe vytvorí nový vrchol daného typu na mieste pustenia myši. V prípade kliknutia na typ vrcholu z ponuky sa vytvorí daný vrchol v plátne na náhodnom mieste. Komponent má definovanú statickú veľkosť a nie je závislý od rozlíšenia prehliadača.

Graf: Tento komponent predstavuje hlavnú časť aplikácie. Nesie v sebe plátno grafu, do ktorého sa následne umiestňujú jednotlivé objekty grafu.

Editor formúl: Tento komponent slúži na vyhodnocovanie zadaných formúl. Komponent obsahuje tlačidlo, ktorým je možné pridať novú formulu. Po pridaní novej formuly sa v tomto komponente vytvorí menší komponent, ktorý obsahuje textové pole, tlačidlo na zmazanie, výberovú ponuku a prvok predstavujúci odozvu. Do textového poľa sa vpisuje formula, ktorá sa vyhodnotí. Formula sa vyhodnotí pri každej zmene textového poľa, resp. pri zmene hodnoty vo výberovej ponuke. Výberová ponuka slúži pre používateľov na určenie spätnej odozvy, či má byť formula splniteľná alebo nie. Ak napríklad vo výberovej ponuke zvolíme, že formula má byť splniteľná a formula nie je splniteľná, tak dostaneme odozvu, že to, čo chceme aby nastalo neplatí. Ak by ale formula bola splniteľná a vyberieme v ponuke, že má byť splniteľná, tak to platí. Táto ponuka má aj neučený stav, ktorý neposkytuje spätnú odozvu. Tento komponent môže obsahovať viacero takýchto formúl. Na konci tohto komponentu sa nachádza tlačidlo na pridanie novej formuly.

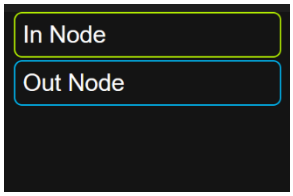
Editor termov: Tento komponent funguje podobne ako editor formúl, avšak namiesto vyhodnocovanie formúl sa tu vyhodnocujú termy. V ponuke výberu sa namiesto možností spĺňania, nespĺňania nachádzajú prvky domény.



Obr. 6.12: Návrh používateľského rozhrania

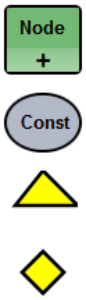
6.5.1 Návrh pridávania vrcholov do grafu

Pridávanie objektov do grafu bolo inšpirované existujúcimi grafickými nástrojmi. V ľavej časti grafu sa vyhradilo miesto na panel, ktorý slúži na pridávanie vrcholov do grafu. Vrcholy bolo spočiatku možné pridávať do grafu iba ťahaním, ale neskôr som pridal možnosť vytvárať ich klikaním. Pôvodný vzhľad tlačidiel panelu grafu bol prevzatý z React Diagrams, ale nepredstavoval skutočný vzhľad jednotlivých typov vrcholov (viď obr. 6.1).



Obr. 6.13: Pôvodný vzhľad tlačidiel na pridávanie vrcholov

Vzhľad tlačidiel som zmenil tak, aby reprezentovali konkrétny typ vrcholu. Teda udalosť na pridanie vrcholu do grafu pridá vzhľadovo rovnaký vrchol ako je vyobrazený na tlačidle. Vzhľad tlačidiel by mal byť robený tak, že berie štýl z vrcholov grafu. Okrem toho by bolo dobré zabezpečiť, aby sa zmena štýlu typu vrcholu okamžite prejavila vo vzhľade tlačidla. Takto by nebolo nutné meniť kód na viacerých miestach, čím by bol kód zároveň zbavený duplicít.



Obr. 6.14: Finálny vzhľad tlačidiel na pridávanie vrcholov

Názvy doménových vrcholov a vrcholov konštánt musia byť unikátne v rámci svojho typu (teda nemôžu existovať dva vrcholy konštanty s rovnakým menom). Pri vytvorení vrcholu v grafe dostane vrchol generické meno. Názov je tvorený počiatočným písmenom typu vrcholu v anglickom jazyku (napríklad pre konštantu to je C) a ďalej je k písmenu priradený index. Index začína od nuly a zvyšuje sa dokiaľ nie je dané meno voľné.

6.6 Import/Export

Aplikácia dovoľuje importovať a exportovať časti jej stavu. Z matematického pohľadu je možné exportovať jazyk, štruktúru, ohodnotenie premenných, splnenie formúl a hodnoty termov. V rámci grafového pohľadu sa exportujú pozície vrcholov. Exportujú sa pozície iba tých vrcholov, ktoré niečo ovplyvňujú v matematickom pohľade. Napríklad ternárny vrchol, ktorý neobsahuje žiaden symbol, resp. nie sú určené všetky jeho parametre, nie je nijak reprezentovaný v matematickom pohľade. Teda súradnice takéhoto vrcholu nie sú exportované.

7 Implementácia

V tejto kapitole bližšie priblížim, ako boli jednotlivé časti aplikácie implementované.

7.1 Použitý jazyk

Aplikácia je primárne určená na počítačové webové stránky a kód sa vykonáva na strane klienta. Je napísaná v jazyku JavaScript, ktorý používam spolu s frameworkom React (táto knižnica je bližšie opísaná v časti 2.2).

Pri výbere JavaScriptového frameworku sme sa rozhodovali medzi Reactom a Angularom. V čase výberu frameworku som nemal žiadne skúsenosti ani s jedným z nich. Pôvodná aplikácia bola naprogramovaná prostredníctvom Reactu, a preto sme sa so školiteľom zhodli, že bude lepšie moju aplikáciu naprogramovať práve pomocou tejto knižnice. Prepísanie pôvodnej aplikácie do iného frameworku by pravdepodobne zabralo viac času, ako pochopenie a úprava už pôvodného kódu.

React sa používa na vytvorenie používateľských rozhraní jednoduchšou cestou. Jednotlivé časti používateľského rozhrania predstavujú komponenty. Každý komponent si uchováva svoj vlastný stav, čo je pri väčšej aplikácie horšie udržateľné. Preto zároveň využívam ďalšiu knižnicu s názvom Redux, ktorá slúži na zhromaždenie stavu premenných (knižnica je bližšie opísaná v časti 2.3). Pomocou tejto knižnice vytvorím objekt, v ktorom sa uchováva stav premenných. Stav je potom dostupný vo viacerých komponentoch.

7.2 Refaktorizácia existujúcej aplikácie

V prvej fáze vývoja bola časť kódu aplikácie refaktorizovaná pre zlepšeniu jeho štruktúry. V pôvodnom kóde bolo mnoho duplicitných komponentov, pričom niektoré z nich sa dali zjednotiť. Okrem toho boli časti kódu veľmi veľké a neprehľadné. Veľa komponentov bolo vytváraných na jednom mieste, pričom obsahovo nezapadali do celku a mohli skôr tvoriť samostatný komponent, ktorý by zhlukoval viacero takto podobných komponentov. Jednou z takýchto častí bol vstupný komponent `App.js` (ktorý sa teraz volá `App.tsx`). Tento komponent bol príliš veľký a kód v ňom bol nečitateľný. Napríklad tlačidlá navigačného panelu netvorili dokopy jeden komponent, ktorý potom stačilo pridať do `App.js` ako menší komponent, ale všetky tlačidlá boli priamo vytvorené v `App.js`. Ďalším príkladom je definícia stavu aplikácie (Kód 7.1), ktorá sa celá nachádzala na jednom mieste . Časti stavu ale nie je nutné definovať len v rámci jedného súboru, a preto som definície jednotlivých častí stavu popresúval do korešpondujúcich súborov (Kód 7.2).

7.3 Migrácia na novší Bootstrap

Prieskumník nie je jedinou aplikáciou vytvorenou pre predmet Matematika 4 – Logika pre informatikov. Existuje viacero takýchto aplikácií, pričom je cieľom ich všetky zjednotiť pod jeden systém. Aby prechod pod jednotný systém bol čo najjednoduchší, je nutné aby štýl aplikácií bol zjednotený. Preto aj jednou z požiadaviek školiteľa bolo aktualizovať knižnicu Bootstrap.

Kód pôvodnej aplikácie využíval starú verziu Bootstrapu. Nová verzia Bootstrapu v sebe zahŕňala zásadné zmeny vo fungovaní knižnice a bolo nutné zmeniť množstvo elementov stránky. Niektorým elementom, ktorým sa zmenilo iba meno, stačilo nájsť ekvivalent. Avšak niektoré elementy boli úplne odstránené z knižnice, čo spôsobilo rozbitie veľkej časti stránky. Okrem toho bol štýl stránky prispôsobený vlastným štýlom. Tento štýl som úplne vymazal a teraz takmer celý zodpovedá Bootstrapovskému štýlu. V novej verzii robím iba malé zmeny v štýle, ako sú napríklad zmeny okrajov tlačidiel, ktoré teraz majú rovné hrany, vypnutie tieňov

pre tlačidlá, resp. v mobilnej verzii aplikácie (teda pri malom rozlíšení) zmizne text tlačidiel (ostanú iba ikony).

7.4 Organizácia kódu do balíkov

Kód je rozdelený do viacerých balíkov, pričom každý z nich samostatne predstavuje nejaký celok. Väčšina balíkov pôvodnej aplikácie bola presunutá do zložky `math_view` a zároveň sa zmenila aj vnútorná štruktúra tejto zložky (zmenili sa názvy, rozdelenie súborov do zložiek a pod.). V zložke `/src/` sa nachádzajú všetky zdrojové súbory potrebné pre funkčnosť aplikácie.

- `graph_view/` – balík obsahujúci všetky zdrojové kódy týkajúce sa grafu. V tomto balíku sa nachádzajú zložky:
    - `components/` – balík, ktorý obsahuje React komponenty patriace do používateľského rozhrania grafu.
    - `labels/` – balík, ktorý obsahuje zdrojové súbory pre model označenia.
    - `links/` – balík pre model uzlu.
    - `nodes/` – balík, v ktorom sa nachádzajú jednotlivé modely vrcholov (doménový vrchol, vrchol konštanty, n-árne vrcholy).
- `math_view/` – balík, ktorý obsahuje zdrojové súbory matematického pohľadu. Skladá sa z balíkov:
    - `buttons/` – balík obsahujúci React komponenty tlačidiel matematického pohľadu a navigačného panelu.
    - `components/` – obsahuje hlavné časti používateľského rozhrania matematického pohľadu.
    - `components_parts/` – obsahuje menšie komponenty pre hlavné časti komponentov v `components/`.
    - `constants/` – konštantné atribúty, využívané v rámci matematického pohľadu.
    - `model/` – zdrojové súbory dátového modelu matematického pohľadu aplikácie.
- `parser/` – obsahuje zdrojové súbory pre parser, používané na kontrolu zadaných hodnôt.
- `redux/` – balík, v ktorom sa nachádzajú Redux akcie, kontajnery a reducery.
- `tests/` – balík obsahujúci testy pre parser a matematický pohľad.

Priečinok `/public/` obsahuje súbory, ktoré sa týkajú štýlu a zároveň obsahuje hlavnú štruktúru webovej stránky (súbor `index.html`). Nachádza sa tu štýl knižnice Bootstrap. Okrem Bootstrap štýlu sa tu nachádzajú zdrojové súbory pre Font Awesome ikony. Obidve tieto knižnice sú bližšie opísané v časti 2.5. Posledným štýlom nachádzajúcim sa v tomto súbore je vlastný štýl, ktorým som stránku dodatočne upravil.

7.5 Organizácia komponentov a kontajnerov

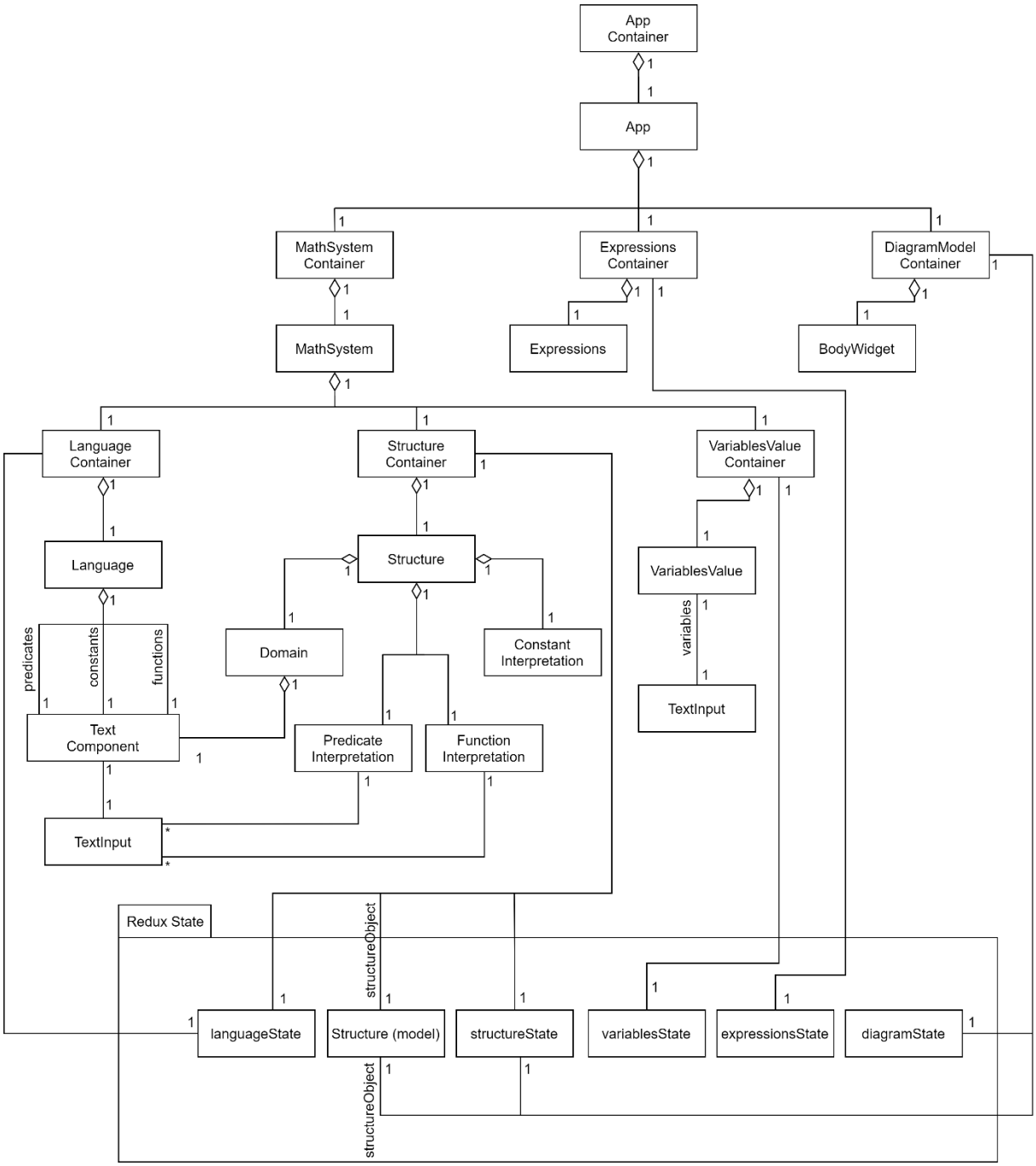
Vstupný komponent aplikácie je `index.tsx`, v ktorom je vytvorený `store` pre aplikáciu. Následne je `store` poslaný cez props do kontajnera `AppContainer`.

`AppContainer` zaobaľuje komponent `App`. Tento komponent obsahuje základnú štruktúru stránky, ktorá sa skladá z navigačného panela, matematického a grafového pohľadu. Všetky komponenty v rámci `App` sú zaobalené do komponentu `Provider`, ktorý zabezpečuje, že ostatné komponenty majú prístup ku stavu aplikácie. Na tomto mieste sa nachádza lokálna stavová premenná `diagramToggled`, ktorá rozhoduje, ktorý z pohľadov sa má zobraziť. V

prípade grafového pohľadu sa renderuje `DiagramModelContainer`, zatiaľ čo pre matematický pohľad to je `MathSystemContainer`.

`DiagramModelContainer` zaobaluje `BodyWidget`, ktorý predstavuje grafový pohľad. Zabezpečuje pre graf všetky potrebné akcie a údaje zo stavu. Ostatné kontajnery sú súčasťou matematického pohľadu. Hlavným kontajnerom pre matematický pohľad je `MathSystemContainer`, ktorý zaobaľuje `MathSystem`. `MathSystem` v sebe nesie kontajnery `LanguageContainer`, `VariablesValueContainer` a `StructureContainer`.

Nakoniec v rámci `App` komponentu sa renderuje `ExpressionsContainer`, ktorý zaobaľuje `expressions`.



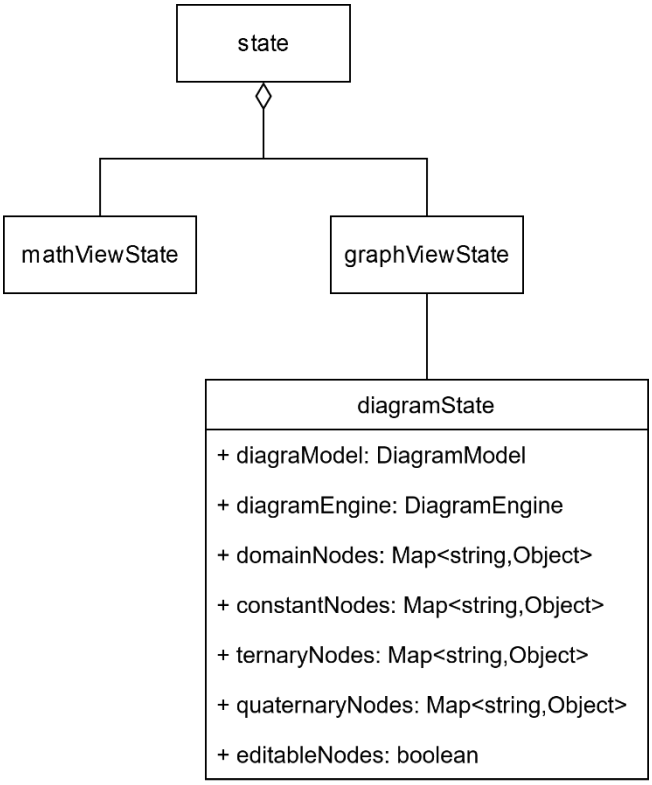
Obr. 7.1: Class diagram komponentov a kontajnerov

Hlavné komponenty grafového pohľadu sa nachádzajú v súbore `graph_view/components/`. Komponenty jednotlivých objektov grafu sa nazývajú widgety. Widgety sa nachádzajú medzi zdrojovými súbormi konkrétnych grafových objektov (napríklad pre vrchol konštanty to je `graph_view/nodes/constant/`).

7.6 Organizácia stavu aplikácie

V nasledovnej sekcii opíšem ako vyzerá stav aplikácie. Okrem toho spomeniem, ako je aktualizovaný vzhľad vrcholov pri zmene ich stavu. Stav matematického pohľadu je bližšie opísaný v sekcii 4.2.

V aplikácii som využil knižnicu Redux, ktorá slúži na zjednotenie údajov aplikácie. Stav aplikácie sa skladá z dvoch častí, z `mathViewState` a `graphViewState`.



Obr. 7.2: Redux stav aplikácie

Matematický stav: predstavuje stav matematického pohľadu. Bližšie je vysvetlený v sekcii 4.2.

Grafový stav: predstavuje stav grafového pohľadu. Ide o objekt, v ktorom sú uložené informácie o celom grafe. Premenná `diagramModel` obsahuje referencie všetkých objektov grafu. Premenná `diagramEngine` predstavuje engine grafu a nesie informáciu ohľadne canvasu. Premenné `domainNodes`, `constantNodes`, `ternaryNodes` a `quaternaryNodes` sú mapy, kde kľúčom je meno vrcholu, ktoré je unikátne v rámci daného typu vrcholu a hodnotou je referencia na objekt vrcholu. Premenná `editableNodes` určuje, či sa všetky vrcholy nachádzajú v pohybujúcom (keď hodnota `editableNodes` je false) alebo editovacom režime (keď hodnota premennej `editableNodes` je true). Každý z vrcholov si zároveň uchováva svoj vlastný stav.

7.6.1 Štruktúra stavu aplikácie

Oproti pôvodnej aplikácii bol stav matematického pohľadu prerobený (refaktorizovaný), aby práca s ním bola jednoduchšia a zároveň bol k nemu pridaný stav grafového pohľadu. Pôvodný (matematický) stav vyzeral nasledovne:

```
const defaultState = {
  structureObject: new Structure(new Language()),
  common: {
    teacherMode: false
  },
  language: {
    constants: defaultInputData(),
    predicates: defaultInputData(),
    functions: defaultInputData(),
  },
  structure: {
    constants: {},
    predicates: {},
    functions: {},
    variables: {...defaultInputData(), object: new Map()},
    domain: {...defaultInputData(), errorMessage: EMPTY_DOMAIN},
  },
  expressions: {
    formulas: [],
    terms: []
  }
};
```

Kód 7.1: Pôvodná štruktúra state

Ide o komplexný stav, v ktorom sa používajú viaceré reducere, pričom každý z nich ovláda nejakú časť stavu. Presnejšie ide o časti `structureObject`, `common`, `language`, `structure` a `expressions`. Všetky tieto časti boli v pôvodnej aplikácii definované na jednom mieste. Pre zlepšenie štruktúry kódu som definovanie častí stavov presunul do korešpondujúcich súborov, teda napríklad definovanie `language` sa nachádza v `language.js` a pod. Kód je týmto čitateľnejší a dá sa v ňom ľahšie orientovať. Kód stavu som upravil nasledovne:

```
const defaultState = {
    structureObject: new Structure(new Language()),
    common: {
        teacherMode: false
    },
    language: languageDefaultState(),
    structure: structureDefaultState(),
    expressions: expressionsDefaultState(),
    diagramState:diagramDefaultState()
};
```

Kód 7.2: Upravená, finálna štruktúra state

7.6.2 Aktualizovanie objektov grafu pri zmene ich stavu

Nakoľko má editor zvládať zobrazovať veľké množstvo objektov v plátne, tak je naprogramovaný tak, aby sme v ňom dosahovali čo najlepší výkon. Objekty grafu nie sú preto prekresľované pri každej zmene, ale iba ak nastane jeden zo spomenutých prípadov:

a) Ak je vypnutá optimalizácia grafu, teda všetky elementy grafu sa prekresľujú pri hociktorej zmene.
b) Ak sa objekt momentálneho modelu nerovná tomu nasledovnému.
c) Ak nastane zmena v parametroch modelu objektu.

V mojej aplikácii využívam na aktualizovanie grafu zmeny parametrov modelu objektu. Mnou vytvorené modely v sebe obsahujú aj vlastné atribúty, ktoré aj keď sa zmenili, tak objekty sa neprekresľovali. Bolo nutné tieto atribúty pridať do funkcie `serialize`, čím som zabezpečil aktualizovanie modelov aj v prípade mnou vytvorených atribútov. V kóde som na viacerých miestach využíval typ `Set`, pričom pri jeho zmene, teda po pridaní alebo odobratí prvku z množiny sa vrchol neaktualizoval i keď sa práve mal. Dôvod bol ten, že sa porovnával objekt, ktorý bol rovnaký aj po pridaní alebo odobratí prvku. Jedným z riešení bolo prechádzať celú množinu a porovnať ju z predošlou, resp. porovnať počet krokov. Keďže som toto potreboval aj pre iné typy, tak som sa rozhodol, že najlepšie bude použiť premennú, ktorá bude slúžiť iba na aktualizovanie stavu. Preto som vytvoril premennú `changeCounter`, typu `number`, ktorej hodnota sa pri každej potrebnej akcii zvýšila o jedna.

```
serialize() {
   return {
      ...super.serialize(),
      changeCounter: this.changeCounter,
      appendPort: this.appendPort,
      editable: this.editable
   };
}
```

Kód 7.3: Upravená `serialize` funkcia

7.6.3 Akcie a reducery

Moja aplikácia využíva akcie a reducery pôvodnej aplikácie. K nim som následne pridal nové, ktoré sú určené na prácu so stavom grafu. Akcie sú uložené v súbore `redux/actions/index.js` a reducery sú v zložke `redux/reducers/`.

Pridané akcie sa vykonávajú vo viacerých reduceroch (akcie na prácu s objektami grafu a akcie na prácu so symbolmi). Vstupný reducer sa nachádza v `index.ts`. Reducer, ktorý sa stará o stav grafu je umiestnený v `diagram.js`. Akcie grafu je možné rozdeliť do nasledovných skupín:

Akcie na prácu s objektami grafu: Patria sem akcie `{add, remove}{Domain, Constant, Ternary, Quaternary}Node`, ktoré slúžia na pridanie a odobratie vrcholu z grafu a `rename{Domain, Constant}Node`, ktorou sa premenováva názov doménového alebo konštantového vrcholu.

Akcie na prácu so symbolmi: Patria sem akcie, ktoré slúžia na synchronizovanie symbolov v zozname vrcholov so symbolmi v matematickom pohľade. Sú to napríklad `add{arity}{symbols}`, `remove{arity}{symbols}` kde arity je prvok z množiny `{Unary,Binary,Ternary,Quaternary}` a `symbols` predstavuje prvok z množiny `{Predicate,Function}`. Akcia `changeDirectionOfBinaryRelation` sa používa na zmenenie smeru binárneho vzťahu (binárneho predikátu alebo unárnej funkcii).

Akcie slúžiace na prepínanie pohľadov: Akcia `syncDiagram` sa volá pri prepínaní matematického pohľadu na grafový. Zabezpečuje synchronizácia grafu s matematikou. Akcia `syncMathState` slúži na zmazanie označení z grafu po prepnutí na matematický pohľad. Táto akcia je nutná, pretože inak označenia spôsobovali chyby v aplikácii. Akcia `importApp` nastaví grafu nový model. Akcia `importDiagramState` slúži na importovanie pozícií vrcholov z importovaného cvičenia.

Všeobecné akcie: Patrí sem akcia `clearGraphSelection`, slúžiaca na zrušenie označenia (všetkých) objektov grafu.

8 Testovanie so študentami

Po dokončení implementácie prebehlo testovanie aplikácie za pomoci študentov z predmetu Matematika 4.

8.1 Zámer testovania

Hlavným zámerom testovania bolo zistiť, ako aplikácia pôsobí na študentov (testerov) a či funguje podľa ich očakávaní. Testovanie malo zároveň pomôcť odhaliť chyby, ktoré neboli zistené počas vývoja aplikácie. Výsledky testovania mali pomôcť určiť, ktoré časti aplikácie treba doladiť.

8.2 Priebeh testovania

Študenti mali možnosť pri domácej úlohe z predmetu Matematika 4 – Logika pre informatikov využiť prieskumník, ku ktorému dostali dotazník ohľadne jeho použiteľnosti. Avšak napriek našim očakávaniam sa našli iba traja študenti, ktorí vyplnili dotazník, pričom tieto odpovede ešte neboli dostatočne konkrétne. Kvôli tomu bolo nutné zmeniť spôsob testovania.

Zorganizovali sme teda testovanie použiteľnosti prieskumníka s dobrovoľníkmi. Testovanie prebehlo formou individuálnych video hovorov. Každý dobrovoľník riešil konkrétnu úlohu z logiky (príklad sa nachádza ako príloha v časti označenej Príloha B), pričom sme sa hlavne zamerali na prácu s jednotlivými typmi vrcholov a určovanie predikátových a funkčných symbolov pomocou grafu. Vďaka tomuto spôsobu testovania sme mohli vidieť študentov priamo pri práci s prieskumníkom, pričom sme dostali okamžitú spätnú väzbu.

8.3 Výsledky testovania – chyby

Počas testovania boli zistené tri chyby v aplikácii. Prvé dve chyby som zistil sám počas testovania. Prvá chyba spôsobovala, že pri importovaní cvičenia sa vytvorené binárne vzťahy nesynchronizovali. Druhá chyba sa tiež týkala importovaných cvičení. Po importovaní údajov nezodpovedal momentálny režim grafu s režimom samotných vrcholov. Tretia chyba bola zistená počas testovania so študentom. Stlačenie tlačidla Delete alebo Backspace spôsobovalo zmazanie označených objektov grafu aj v prípade, keď kurzor myši bol nastavený na textové pole formuly alebo termu. Tieto chyby boli opravené a na web bola nahratá nová verzia aplikácie.

8.4 Výsledky testovania – spätná odozva

Počas individuálneho testovania sme dostali cennú spätnú väzbu k prieskumníkovi. Osobne som očakával, že práca s grafom pôjde študentom ťažšie. Napriek predpokladom všetci testovaní rýchlo pochopili základy fungovania grafu (ako je prácu s režimami grafu, pridávanie vrcholov do grafu, importovanie cvičenia). Žiadnemu z testovaných neprekážalo používanie režimov grafu a prišli naňho intuitívne bez pomoci.

Menšie komplikácie vznikali pri vytváraní vrcholov, kedy niektorí z testerov nevedeli, ktorý typ vrcholu má akú funkcionalitu. Spočiatku bolo problémom zistiť, ktorý z vrcholov predstavuje prvok domény. Tento problém vznikal hlavne z toho, že grafový pohľad predstavuje iný pohľad na štruktúry, ako matematický.

Najväčšia komplikácia bola pri vytváraní binárneho vzťahu, ktorý fungoval iným štýlom ako ternárne alebo kvaternárne vzťahy. Na vyjadrenie ternárneho a kvaternárneho vzťahu slúžia typy vrcholov, zatiaľ čo binárny vzťah vzniká vytvorením hrany medzi doménovými prvkami. Podľa testerov by mali byť binárne vzťahy vyjadrené rovnakou cestou, teda pomocou vrcholov, čím by bola zachovaná konzistencia vo vytváraní vzťahov v grafe.

Jeden zo študentov sa vyjadril, že by bolo lepšie, ak by boli funkčné symboly v zozname vrcholu vyznačené inou farbou ako sú predikáty. Toto by sa prejavilo hlavne pri väčšej štruktúre, v ktorej už nie je úplne jasné, ktorý symbol je akého typu.

Okrem spomenutých komplikácií sme sa pri testovaní nestretli s väčšími problémami. Väčšina študentov sa vyjadrila, že práca s grafom bolo intuitívna. Študenti pochválili využitie grafu ako nový spôsob na zobrazenie, tvorenie a editáciu štruktúr. Taktiež sa jeden zo študentov vyjadril, že mu práca s grafom prišla intuitívnejšia ako matematický pohľad, do ktorého je nutné písať text.

8.5 Záver testovania

Dojmy z testovania boli pozitívne. Až na menšie nedostatky boli študenti s aplikáciou spokojní. Vzniknuté nejasnosti, akou je napríklad to, že binárne vzťahy fungujú iným spôsobom ako vzťahy ostatných arít vznikali aj preto, lebo si študenti neprečítali príručku (pomocník) ku grafu. Po odkázaní sa na pomocník nám študenti povedali, že funkcionality grafu sú práve v ňom detailne opísané, a že dodatočné vysvetľovanie už tým pádom nie je nutné. Prerobenie binárnych vzťahov by predstavovalo náročnejšiu zmenu v kóde, a preto sme sa rozhodli, že vzniknutý problém stačí riešiť formou vysvetlenia pomocou pomocníka ku grafu (na ktorý bude kladený väčší dôraz).

Vďaka testovaniu sa mi podarilo odhaliť chyby v aplikácii, pričom všetky z nich boli úspešne vyriešené. Okrem toho sme sa dozvedeli, že graf je dobre nastavený a že vzniknuté komplikácie s používaním grafu sa dajú riešiť inou cestou (vysvetlením).

Záver

Po analýze koncového stavu aplikácie môžem skonštatovať, že požiadavky na aplikáciu boli splnené. Podarilo sa mi navrhnúť spôsob, akým sa zobrazujú štruktúry v grafe. Následne som refaktorizoval kód pôvodného prieskumníka a rozšíril ho o grafový a databázový pohľad. Aplikácia bola testovaná študentami, pričom som na ňu dostal pozitívnu spätnú väzbu. Výsledkom mojej práce je funkčná a stabilná aplikácia.

Počas vývoja tejto aplikácie som sa naučil množstvo nových vecí, týkajúcich sa hlavne tvorby webových a edukačných systémov. Naučil som sa pri nej používať množstvo užitočných a moderných knižníc, ktoré mi môžu neskôr pomôcť v zamestnaní.

Aplikáciu je možné naďalej vyvíjať. Používateľské rozhranie grafu by sa dalo vylepšiť ovládacími prvkami grafu a určite aj lepšími vysvetlivkami k objektom grafu. Z výsledkov testovania sme sa dozvedeli, že by bolo lepšie, ak by binárne vzťahy fungovali rovnakým spôsobom, ako ternárne a kvaternárne vzťahy. Preto by sa ďalšie zlepšenie mohlo týkať zjednotenia binárnych vzťahov s ternárnymi a kvaternárnymi vzťahmi. Okrem toho by sa dala pridať možnosť vytvárať grafy pre vyššie arity (momentálne je maximálna arita 4).

Ďalšie zlepšenie by sa mohlo týkať časti editora formúl a editora termov. Písanie formúl je dosť ťažkopádne, pretože v sebe obsahuje množstvo špeciálnych symbolov a zároveň je od vyhodnocovača formúl požadované, aby bola formula úplne uzátvorkovaná, inak nie je výraz vyhodnotený (aj v prípadoch, kedy zátvorky neovplyvňujú jej význam). Aplikácia by ešte mohla byť vylepšená pre mobilné zariadenia, pretože momentálne je robená pre stolné počítače a niektoré časti aplikácie nefungujú na mobiloch správne (ako je ťahanie vrcholov a pod.). Ako posledné zlepšenie vidím možnosť, že by sa aplikácia integrovala do interaktívneho pracovného hárku, ktorý vznikol paralelne v rámci bakalárskej práce Nikolaja Knihu. Ten zabezpečí odovzdávanie úloh, riešených pomocou rôznych logických výučbových aplikácií.

Literatúra

[1] Milan Cifra. *Prieskumník sémantiky logiky prvého rádu.* Bakalárska práca, Univerzita Komenského v Bratislave, Fakulta matematiky, fyziky a informatiky, 2018.

[2] Ján Kľuka a Jozef Šiška. *Prednášky z Matematiky (4) – Logika pre informatikov*, 2019. Dostupné aj na: https://github.com/FMFI-UK-1-AIN-412/lpi/tree/master/prednasky

[3] Vítězslav *Švejdar. Logika: neúplnost, složitost, nutnost.* Academia, Praha, 2002. Dostupné aj na: http://www1.cuni.cz/~svejdar/book/LogikaSve2002.pdf.

[4] Mozilla and individual contributors. *MDN web docs*, 2020. Dostupné na: https://developer.mozilla.org/en-US/. [Online, cit. 23.01.2020].

[5] WHATWG. *HTML Living Standard*, 2020. Dostupné na: https://html.spec.whatwg.org/. [Online, cit. 30.05.2020].

[6] W3C. *CSS Snapshot 2018*, 2019. Dostupné na: https://www.w3.org/TR/css-2018/. [Online, cit. 30.05.2020].

[7] Facebook Inc. and the React documentation authors. *React*, 2020. Dostupné na: https://reactjs.org/. [Online, cit. 17.01.2020].

[8] Dan Abramov and the Redux documentation authors. *Redux,* 2020. Dostupné na: https://redux.js.org/. [Online, cit. 25.01.2020].

[9] Microsoft Corporation. *Typescript*, 2020. Dostupné na: https://www.typescriptlang.org/. [Online, cit. 27.01.2020].

[10] Bootstrap team and contributors. *Bootstrap*, 2020. Dostupné na: https://getbootstrap.com/. [Online, cit. 10.05.2020].

[11] React Bootstrap team. *React Bootstrap*, 2020. Dostupné na: https://react-bootstrap.github.io/. [Online, cit. 10.05.2020].

[12] Font Awesome. *Font Awesome*, 2020. Dostupné na: https://fontawesome.com/v3.2.1/. [Online, cit. 10.05.2020].

[13] React Diagrams community authors. *React Diagrams,* 2020. Dostupné na: https://projectstorm.gitbook.io/react-diagrams/. [Online, cit. 20.01.2020].

[14] React Digraph community authors. *React Digraph,* 2020. Dostupné na: https://github.com/uber/react-digraph. [Online, cit. 20.01.2020].

[15] Martin Krasňan. *Kolaboratívny grafický editor pre MediaWiki.* Diplomová práca, Univerzita Komenského v Bratislave, Fakulta matematiky, fyziky a informatiky, 2018.

Príloha A - Dotazník

Poprosím vás vyplniť krátky dotazník ku grafovému prieskumníku https://bl96.github.io/structure-explorer/. V prípade nájdenia chýb ma prosím kontaktujte mailom na baluch9@uniba.sk alebo vytvorte issue na https://github.com/bl96/structure-explorer/issues.

1. Ako ste použili grafovú časť prieskumníka štruktúr? (Môžete označiť viac odpovedí.)
   a) Vytvoril som v ňom takmer celú štruktúru.
   b) Vytvoril alebo upravil som v ňom časť štruktúry.
   c) Našiel som si chyby.
   d) Overil som si správnosť môjho riešenia.
   e) Pochopil som niečo, čomu som predtým nerozumel.
   f) Nepoužil som ju.
   g) Iné (napíšte)
   .....

2. Ak vám prieskumník pomohol pochopiť niektorú časť z logiky, tak ktorá to bola?
   .....
   .....

3. Používali ste skôr matematickú (textovú) časť alebo graf? (Vyberte jednu z možností.)
   a) Preferujem skôr dopisovanie do matematickej časti.
   b) Lepšie sa mi pracovalo s grafom ako s matematickou časťou.
   c) Používal som obidve časti asi v rovnakej miere.

4. Bola práca s grafom dostatočne intuitívna? (Vyberte jednu z možností.)
   a) Áno, úplne.
   b) Niektoré veci mi neboli jasné.
   c) Nepochopil som ako sa pracuje s grafom.

5. Ak boli niektoré časti používateľského rozhrania grafu nejasné, tak o ktoré išlo?
   .....
   .....

6. Boli vysvetlivky (nápoveda) ku grafu dostatočné? (Vyberte jednu z možností.)
   a) Áno.
   b) Nie.
   c) Nenašiel som ich.
   d) Nepotreboval som ich.

7. Odporučili by ste aplikáciu ostatným študentom?
   a) Áno.
   b) Nie.

8. Čo by ste zmenili alebo pridali do grafového prieskumníka?
   .....
   .....

Príloha B – Úloha z logiky

Cvičenie 10.1. (3.4.1) Nájdite štruktúru, ktorá splní prvorádovú teóriu $T = \{A_1, \ldots, A_6\}$ v jazyku $\mathcal{L}$, kde $\mathcal{V}_{\mathcal{L}} = \{u, v, w, x, y, z\}$,
$C_{\mathcal{L}} = \{\mathrm{Andrea}, \mathrm{Danka}, \mathrm{Hanka}, \mathrm{Janka}, \mathrm{Max}, \mathrm{Nikita}\}$,
$\mathcal{F}_{\mathcal{L}} = \{\mathrm{manžel}^1, \mathrm{manželka}^1, \mathrm{prvorodené_dieťa}^2\}$,
$\mathcal{P}_{\mathcal{L}} = \{\mathrm{Lekár}^1, \mathrm{Manželia}^2, \mathrm{Muž}^1, \mathrm{Právnik}^1, \mathrm{Žena}^1\}$.

(A_1) $\forall x \forall y (\mathrm{Manželia}(x, y) \rightarrow \mathrm{Manželia}(y, x))$

(A_2) $\forall x \forall y \big(\mathrm{Manželia}(x, y) \rightarrow$
$(\mathrm{Muž}(x) \rightarrow x \doteq \mathrm{manžel}(y)) \wedge (\mathrm{Žena}(x) \rightarrow x \doteq \mathrm{manželka}(y))\big)$

(A_3) $\forall x \forall y \big(\mathrm{Manželia}(x, y) \wedge \mathrm{Muž}(x) \wedge \mathrm{Žena}(y) \rightarrow$
$\mathrm{Lekár}(\mathrm{prvorodené_dieťa}(x, y)) \vee \mathrm{Právnik}(\mathrm{prvorodené_dieťa}(x, y))\big)$

(A_4) $\mathrm{Manželia}(\mathrm{Hanka}, \mathrm{Max}) \wedge \mathrm{Žena}(\mathrm{Hanka}) \wedge \mathrm{Muž}(\mathrm{Max})$

(A_5) $\mathrm{Manželia}(\mathrm{Danka}, \mathrm{Janka}) \wedge \mathrm{Žena}(\mathrm{Danka}) \wedge \mathrm{Žena}(\mathrm{Janka})$

(A_6) $\mathrm{Manželia}(\mathrm{Andrea}, \mathrm{Nikita}) \wedge \mathrm{Muž}(\mathrm{Andrea})$

Príloha C – Elektronická príloha

Zdrojové súbory tejto aplikácie sú dostupné v elektronickej prílohe bakalárskej práce a na adrese: https://github.com/bl96/structure-explorer

Funkčná verzia aplikácie je taktiež dostupná na adrese: https://bl96.github.io/structure-explorer/